

Implementation Notes

What this document is: a complete record of what was built, why each decision was made, and what each change was worth — written for someone with **no background in search or recommender systems**.

Every technical term is explained in plain language, with a real example from this catalog, before it is used. If you hit a word you don't recognise, it is either defined in [§2 Vocabulary](#2-vocabulary) or in the [Glossary](#glossary) at the end.

Each stage section ends with **"Ideas for this stage"** — enhancements specific to that piece of the pipeline. Enhancements that cut across several stages, or that are about how the stages are *orchestrated*, are collected in [§10](#10-cross-cutting-and-orchestration-ideas) at the end.

Result: the supplied starter agent scores **0.1067**. This implementation scores **0.8592** — roughly 8x — using no AI model, no network connection, and nothing outside the Python standard library.

Table of contents

- 1 [What the agent has to do](#1-what-the-agent-has-to-do)
 - 2 [Vocabulary](#2-vocabulary)
 - 3 [How the score is calculated](#3-how-the-score-is-calculated)
 - 4 [Why the supplied baseline scored 0.1067](#4-why-the-supplied-baseline-scored-01067)
 - 5 [Stage-by-stage changes](#5-stage-by-stage-changes)
 - 6 [S0 — Experiment harness](#s0--experiment-harness)
 - 7 [S1 — Catalog index](#s1--catalog-index)
 - 8 [S2 — Intent router](#s2--intent-router)
 - 9 [S3 — Dialog state](#s3--dialog-state)
 - 10 [S4 — Clarification policy](#s4--clarification-policy)
 - 11 [S5 — Retrieval](#s5--retrieval)
 - 12 [S6 — Reranking](#s6--reranking)
 - 13 [S7 — Recommendation timing](#s7--recommendation-timing)
 - 14 [S9 — Robustness and tests](#s9--robustness-and-tests)
 - 15 [What was tried and rejected](#6-what-was-tried-and-rejected)
 - 16 [Results](#7-results)
 - 17 [Why upstream Amazon data was ruled out](#8-why-upstream-amazon-data-was-ruled-out)
 - 18 [Files at a glance](#9-files-at-a-glance)
 - 19 [Cross-cutting and orchestration ideas](#10-cross-cutting-and-orchestration-ideas)
 - 20 [Glossary](#glossary)
-

1. What the agent has to do

There is a **catalog** of 50,000 clothing, shoe and jewellery products (`data/catalog.jsonl`). One of them is the **target** — a specific item a specific customer actually bought. The agent is never told which one.

A **session** is one conversation. It runs for at most **10 turns**. On each turn:

- the customer sends a message,
- the agent replies with a sentence, optionally a *question*, and optionally a *list of up to 10 product IDs*.

The session ends the moment the target appears anywhere in that list — or after turn 10, which counts as a failure.

Here is a real session from the public set, produced by the finished agent:

```
session public_0008
TARGET: Hanes Womens Wireless Bra, Full-Coverage Pullover Stretch-Knit Bra

turn 1  customer> I'm looking for Bras Everyday Bras. A key requirement is: nylon.
        agent   > To narrow this down: is there anything else that matters
                  for this one?                                [asks: other]  no list yet

turn 2  customer> For that, what matters is: 96% Nylon, 4% Spandex; Pull-On closure.
        agent   > (same question)                                [asks: other]  no list yet

turn 3  customer> For that, what matters is: Hand Wash Only.
        agent   > (same question)                                [asks: other]  10 products
                  -> TARGET at position 1.  HIT on turn 3.
```

(That is a real run, reproducible with the agent as it stands. Not every session looks like this — in some, the agent becomes confident enough to show a list on turn 2; §S7 explains when and why.)

Two things in that transcript are the whole story of this project, and both are explained in detail later:

- 1 The customer **only volunteered information because the agent asked a question**. Had the agent not asked, turns 2 and 3 would have contained nothing useful.
- 2 The agent **deliberately showed no products on turns 1 and 2**, even though it could have. §3 explains why holding back is worth more than it costs.

2. Vocabulary

Read this once; everything later depends on it.

The data

Term	Meaning
Catalog	The 50,000 products. One JSON object per line in data/catalog.jsonl.
parent_asin	Amazon's product ID, e.g. B07KCFS4VC. The <i>only</i> thing that is scored — a recommendation is right only if this string matches the target exactly. Close is not counted.
Product fields	title, features (bullet points), description, price, categories, details (a dictionary like {"Department": "Womens"}), average_rating, rating_number, store (the brand).
Session	One conversation with one customer, targeting one product.
Turn	One customer message plus one agent reply. Maximum 10 per session.
Target	The hidden product the agent must find.
Scenario	Sessions come in four flavours: buying (customer states a firm requirement immediately), browsing (customer starts vague), intent override (customer changes their mind partway through), boundary (customer has no opinion on something the agent asks about).

The two halves of any search system

This distinction is the single most important idea in the document, and the architecture is built around it.

Retrieval = finding a shortlist of plausible candidates out of 50,000. **Ranking** = ordering that shortlist so the best one is first.

They are different jobs with opposite requirements:

- Retrieval must be **generous**. If the target isn't in the shortlist, nothing downstream can save you. Retrieval is allowed to include junk.
- Ranking must be **strict**. Its job is to separate the one right answer from 299 near-misses.

A technique can be excellent at one and terrible at the other. §6 documents a case where exactly that happened, and where moving one technique from retrieval to ranking turned a large loss into a large gain.

How text search works (BM25)

The agent finds products by matching words. The scoring function used is **BM25**, a standard text-search formula. You don't need the formula — you need three intuitions:

- 1 **Rare words count more than common words.** If a customer says "kandinsky", that word appears in maybe one product in the whole catalog, so it is enormously informative. If they say "imported", which appears in tens of thousands, it tells you almost nothing. BM25 weighs each word by its rarity across the catalog. (The technical name for this rarity weighting is **IDF**, *inverse document frequency*.)
- 2 **More matches are better, but with diminishing returns.** A product mentioning "leather" five times isn't five times more relevant than one mentioning it once.
- 3 **Which field matched matters.** A word in the product's title is stronger evidence than the same word buried in a long description. This is configurable — see `DEFAULT_WEIGHTS` at `src/index.py:25`.

The search index itself is **SQLite FTS5**, a full-text search engine built into Python's standard library. That is why this project needs no third-party packages: the search engine is already on your machine.

Measuring success

Term	Meaning
Top-k	The first k items of a ranked list. Here $k = 10$.
Hit	The target appeared in a list the agent showed.
Hit Rate@10	Fraction of sessions that ended in a hit. 0.94 = the agent found the target in 94% of sessions.
Rank	The target's position in the list. Position 1 is best.
Reciprocal rank (RR)	$1 \div \text{rank}$. Rank 1 $\rightarrow 1.0$. Rank 2 $\rightarrow 0.5$. Rank 10 $\rightarrow 0.1$. A miss $\rightarrow 0$. This rewards being <i>first</i> far more than merely being <i>present</i> .
MRR	Mean Reciprocal Rank — the average RR across all sessions. 0.79 roughly means "on average the target lands around position 1.3".
MTTC	Mean Turns To Conversion — the average turn number on which the hit happened. A miss counts as turn 11. Lower is better.

Terms specific to this project

Term	Meaning
Facet	A structured attribute of a product with a limited set of values — material (leather, cotton), colour (black, navy), price band, category. Contrast with free text. Extracted by <code>src/facets.py</code> .

Constraint span	A short phrase the customer said, normalised for matching — e.g. "stainless steel band". §S6 explains why these are so powerful here.
ask_attribute	A structured field in the agent's reply naming <i>what</i> it is asking about (material, color, budget, other, ...). Critically, this — not the English sentence — is what the customer's simulator reads.

3. How the score is calculated

All of this lives in `evaluator/local_evaluator.py`, which is organizer-owned and **must never be modified**.

Step 1 — *cleaning the agent's list*

`normalize_recommendations` (`evaluator/local_evaluator.py:95`) silently discards:

- IDs that aren't in the catalog,
- duplicates,
- anything after the first 10.

"Silently" matters: a malformed recommendation isn't an error, it just quietly wastes a slot.

Step 2 — *checking for a hit*

```
if override_applied and target in ranked:
    best_rank = ranked.index(target) + 1
    hit_turn = turn
    break
```

— `evaluator/local_evaluator.py:252`

Note the break. **The session stops at the first hit.** This has a consequence that shapes the entire agent design, covered below.

Step 3 — *the four metrics*

From `metric_summary` (`evaluator/local_evaluator.py:188`) and line 280:

```
HitRate@10 = hits / N
MRR        = average(1 / rank), a miss contributes 0
MTTC       = average(hit turn), a miss counts as turn 11
Efficiency = (11 - MTTC) / 10, clamped to the range 0..1

Score = 0.50 x HitRate@10 + 0.30 x MRR + 0.20 x Efficiency
```

For this implementation:

```
0.50 x 0.940 + 0.30 x 0.7911 + 0.20 x 0.7595 = 0.8592
```

The five consequences that drive every design decision

1. The first list containing the target freezes both rank and turn. Because of that break, you never get a second chance to rank better. If you show a list on turn 2 and the target happens to be sitting at position 9, you have permanently locked in $RR = 0.11$ — even though by turn 4 you might have ranked it first. This is the single most important strategic fact in the challenge.

2. Wrong recommendations are free. There is no penalty for showing 10 irrelevant products. The only cost is the turn. So the dangerous failure mode is not *being wrong* — it is *being confident too early*.

3. ask_attribute is the only tap for information. The simulated customer (`customer_reply`, `evaluator/local_evaluator.py:166`) reveals a new constraint **only** when the agent's reply sets `ask_attribute`. If

it is null, the customer answers "ask me about one specific attribute" and discloses nothing. An agent that never asks never learns anything.

4. An exception is an invisible failure. The evaluator wraps each turn in `try / except Exception` (evaluator/local_evaluator.py:241) and converts a crash into an empty response. You lose the session and see no error message.

5. Override sessions cannot convert early. For intent-override scenarios, `override_applied` stays `False` until the customer's change of mind arrives on turn 3 or 4. Hits before that are ignored entirely, which puts a hard floor under that scenario's MTTC.

What each improvement is actually worth

With 200 sessions, here is the score change from improving a *single* session:

Change to one session	Score change
Turn a miss into a hit (rank 5, turn 4)	+0.0035
Move an existing hit from rank 10 to rank 1	+0.00135
Reach the same hit one turn sooner	+0.0001

Read those numbers together and they give a clear instruction:

Converting a miss is ~2.6x more valuable than fixing a rank, and fixing a rank is ~13x more valuable than the turn it costs to earn. Therefore: spend turns freely to gather information; never rush a list out.

Almost every design choice in §5 follows from this one table. It is why the agent stays silent for the first two turns.

4. Why the supplied baseline scored 0.1067

The original starter/agent.py was about 100 lines. Its `respond()` did three things: tokenise **the current message**, run one BM25 query, return the top 10. It scored:

Hit Rate@10	0.125
MRR	0.068
MTTC	9.81
Score	0.1067

Three flaws, in order of cost:

It never asked a question. `ask_attribute` was hard-coded to `None`. By consequence #3 above, the customer therefore never disclosed anything. The agent's entire knowledge of the customer was the opening sentence — for all ten turns.

It was stateless. Even if the customer *had* said something useful, `respond()` only ever looked at `user_message`, the current turn. Everything said earlier was discarded.

It never reranked. Whatever order BM25 returned was the order shown.

The MTTC of 9.81 tells the story: out of 10 possible turns, the average session burned 9.81 of them. The agent was effectively making the same guess ten times in a row, because nothing ever changed between turns.

The key realisation: the biggest available win was not a better search engine. It was *asking a question and remembering the answer*. Fixing only that — before touching retrieval or ranking at all — took the score from **0.1067** to **0.7811**.

5. Stage-by-stage changes

The agent is split into one module per job, under `src/`. `starter/agent.py` remains the file the evaluator imports; it is now a thin adapter that wires the stages together and owns the response format.

Reading order for the code: `starter/agent.py` -> `src/state.py` -> `src/retrieval.py` -> `src/rerank.py`.

S0 — Experiment harness

File: `tools/sweep.py`

What this stage does

It isn't part of the agent. It is the tool used to *measure* the agent — run several configurations and print a comparison table.

Why it was needed

Two problems made experimentation painful.

Speed. Building the search index over 50,000 products takes about 4 seconds, and the original setup rebuilt it for every configuration tested. Comparing four ideas meant four rebuilds, and early runs timed out at two minutes.

Self-deception. There are only 200 public sessions, but **800 private sessions decide the final result**. If you tune ten parameters against 200 sessions and keep whatever scores highest, you will reliably pick settings that fit the noise in those particular 200 — and they will not transfer. This is called **overfitting**.

What changed

Shared index. The index is now cached per catalog path (`load_index` in `src/index.py`), so constructing many Agent objects builds it once. A full 200-session evaluation dropped from timing out to **~13 seconds**.

Dev/holdout split. `split_samples()` divides the 200 sessions into 120 **dev** and 80 **holdout**, stratified so each split keeps the same mix of scenarios. All tuning happens on dev. Holdout is looked at only to confirm a change survives on data it wasn't tuned against. Where dev and holdout disagreed, holdout won — twice, this changed what shipped (see §6).

The split is deterministic (sorted by `sample_id`), so it needs no stored file and never drifts.

```
python3 tools/sweep.py --split dev
python3 tools/sweep.py --split holdout
python3 tools/sweep.py --split all --configs floor,rerank,infogain
```

A probe must accept whatever the agent passes it. `tools/observe.py` and `tools/stress_observe/` trace a session by temporarily replacing `retrieve()` and `rerank()` with wrappers that record timings and candidate pools. Those wrappers had fixed signatures written before the agent grew extra keyword arguments, so every call raised `TypeError` — which the agent's own catch-all handler swallowed, returning an empty answer for the turn. The traced session then scored `0.000` while looking like a legitimate result. Worse, the empty answer made the diagnostic report the target as `never_retrieved` — "never entered the candidate pool, a recall problem" — which is a conclusion about the *retrieval stage*, drawn entirely from a broken tracing tool. Both wrappers now accept and forward arbitrary keyword arguments, so they cannot drift out of step with the agent again. Aggregate scores from `tools/stress_harness.py` were never affected: it does not install these wrappers.

The general lesson is the one this stage exists to enforce — **a measurement tool that fails silently is worse than no tool**, because it produces numbers that look real.

Measured effect

No direct score. Every measurement in this document exists because of it.

Ideas for this stage

- **Confidence intervals.** With 80 holdout sessions, a difference under about 0.02 is indistinguishable from noise. The harness reports point estimates only. Bootstrap resampling over sessions would print an interval, turning "is this better?" from a judgement call into a measurement.
- **Per-session diffing.** The harness compares aggregates. A mode that lists which *specific* sessions changed between two configurations would make regressions far faster to diagnose — currently that requires an ad-hoc script.
- **Automated parameter search.** Several constants were swept by hand (§S7, §S4). A small random or grid search driven from the config matrix would cover more ground, though it raises overfitting risk and would need the confidence intervals above to be safe.
- **Cache the catalog parse.** `catalog_index()` re-parses 58MB of JSON per harness invocation (~2s). A pickled cache would speed up repeated runs from the shell.

S1 — Catalog index

Files: `src/index.py`, `src/text.py`

What this stage does

Reads the 50,000-product catalog once and builds the structures every later stage queries: a full-text search index, and a trimmed copy of each product's data.

Why it was needed

The original code built its index inline inside the agent and kept nothing else, so no other component could ask questions like "what colour is this product?" — the information was inside the search engine and unreachable.

What changed

One pass, three products. `CatalogIndex._build()` walks the file once and produces:

- 1 **The FTS5 full-text index** — seven columns (title, categories, features, details, store, description), so field-specific weighting is possible.
- 2 **Trimmed product records** — `parent_asin`, title, categories, brand, price, ratings, and a normalised text blob. Deliberately not a second full copy of the 58MB file.
- 3 **Normalised text** — this one is subtle and it matters. Product text is stored with all punctuation stripped and words joined by single spaces:

```
"Hide & Drink, 100% Leather, Buckle closure"
    becomes
"hide drink 100 leather buckle closure"
```

The reranker (§S6) checks whether a phrase the customer said appears inside a product's text. Without matching normalisation on both sides, "100 leather" would never match "100% Leather" and the strongest signal in the system would silently do nothing.

Shared caching. `load_index()` returns one index per catalog path, so repeated Agent construction is nearly free.

Word handling (`src/text.py`). Two lists of words are removed before searching:

- *Stopwords* — conversational filler: "the", "for", "looking", "preference".
- *Boilerplate* — Amazon catalog filler: "imported", "closure", "machine", "wash", "dimensions". These appear in tens of thousands of products, so they add query cost without discriminating.

Measured effect

Index build **4.13s**, one time at startup. Peak memory **~282MB**. Per-turn latency **16.6ms mean, 19.8ms at the 95th percentile**.

Ideas for this stage

- **Tune the field weights.** `DEFAULT_WEIGHTS` at `src/index.py:25` is `(0.0, 6.0, 4.0, 2.5, 2.5, 1.5, 1.0)` — inherited verbatim from the supplied baseline and **never tuned**. Those seven numbers control how much a title match counts versus a description match. This is probably the cheapest untapped gain in the repo: the harness exists, the parameter is a single tuple, and nobody has ever checked whether the baseline's guess was any good.
- **Learn the boilerplate list instead of writing it.** ~~Compute document frequency across the catalog and drop the top ~200 terms.~~ **Done, but not the way this originally proposed — the original proposal was wrong.** Ranked by document frequency, polyester is 72nd, cotton 83rd, black 97th, leather 111th and spandex 141st. Dropping the top 200 would delete every one of them, and those are precisely the constraints the customer discloses, because `intent_card()` inserts a material at position 0 and a colour at position 1 of every card (§S1 explains why the customer quotes catalog copy). Meanwhile `asin` — a genuine member of the hand-written list — sits at rank 10,379 with 0.0% document frequency. Frequency fails in both directions.

What works is *where* a token occurs, not how often. Amazon's structural metadata lives in the `details` dict, and those tokens appear almost nowhere else — `department` 100% of its occurrences, `dimensions` 99.6%, `manufacturer` 99.6%, `inches` 97.7% — while attribute values are spread across title, features and description: spandex 1.2%, cotton 2.5%, polyester 3.3%, black 13.4%. The gap between 16% and 96% is empty of real attribute words, so the threshold is read off the catalog's own distribution rather than tuned against a score. `tools/build_stoplist.py` applies that rule and writes `src/stoplist.py`; it reads `data/catalog.jsonl` and never opens `data/public_set.jsonl`, so it cannot fit the public sessions. The learned list reproduces 14 of the 24 hand-written terms and finds 22 more that nobody thought to write down — every month name and year 2014–2022, harvested from "Date First Available: August 15, 2019".

Half the list is not learnable, and `src/text.py` now says so. `imported`, `machine`, `wash`, `closure` and `care` are care-and-origin phrases in features/description. Two statistics were tested and both fail to separate them from real attributes: by document frequency `closure` (38.6% of products) outranks `polyester` (21.8%), and by spread across the 12 largest category buckets `polyester` (CV 0.51) falls between `imported` (0.49) and `wash` (0.60), with `black/white` (0.34/0.31) landing inside the scaffolding band. What separates them is that a shopper does not choose between "imported" and "not imported" — semantics, not frequency. Those ten stay hand-written, with the evidence recorded beside them so the experiment is not retried.

Measured effect: none. Public set identical to six decimal places (0.912526 both ways), adversarial set -0.0001, dev and holdout flat. Boilerplate removal strips a median of one token from a ten-token query and `MAX_QUERY_TERMS = 60` never binds — 0 of 200 sessions come close. It ships for auditability and for robustness on a catalog nobody has hand-inspected, not for points. Measurements were taken in one process with everything else held constant; see `tests/test_stoplist.py` for the invariants that keep a future regeneration honest.

- **Persist the index.** 4.1s of startup could become near-zero by writing the FTS5 index to a file instead of `:memory:`. Only worth it if startup time is ever judged.
- ~~**Dead code.** `search_phrases()` (`src/index.py:119`) has had no caller since the phrase route was removed in §6. It should go.~~ **Done** — deleted, along with `DialogState.query_terms()` (`src/state.py`), which also had no caller. Both deletions verified score-identical: public 0.915887, hard 0.794375, 57/57 tests.

S2 — Intent router

File: `src/router.py`

What this stage does

Classifies the opening message as **buying** (customer knows what they want) or **browsing** (customer is exploring).

Why it was needed

The problem brief calls for "dual-track routing" as a core pillar: a decided customer and an exploring one deserve different treatment.

What changed

Classification uses **linguistic cues**, not exact string matching:

```
BUYING_CUES    = "key requirement | must be | i need | it has to | ..."  
BROWSING_CUES = "still exploring | just looking | not sure | open to | ..."
```

This was a deliberate choice with a trap avoided. The local evaluator builds its opening lines from fixed templates (`initial_message`, `evaluator/local_evaluator.py:154`), so matching those exact strings would score perfectly here — and could fail completely on the private sessions if the organizer paraphrases them. Cue matching degrades gracefully; template matching fails silently.

Unknown phrasing defaults to **browsing**, because mistaking a vague customer for a decided one commits to constraints they never stated, whereas the reverse costs at most one extra question.

Measured effect — and an honest scope reduction

The router was first built to change *retrieval*: a wider candidate pool for browsing (400), a narrower one for buying (200). Measured:

	dev	holdout
Router affects retrieval	0.8715	0.8373
Router does not	0.8715	0.8391

Identical on dev, 0.002 worse on holdout. The reranker (§S6) already resolves both tracks well, so there was nothing left for routing to fix.

Rather than keep a feature that does nothing and describe it as a pillar, the router was **scoped down** to the job it genuinely does: phrasing. A browsing customer hears "To point you in the right direction: ..."; a buying customer hears "To narrow this down: ...". That is a real product requirement even where it is not a scoring one, and the measurement is recorded in the module docstring so the next reader doesn't repeat the experiment.

Branch dual_tracking / stress_harness — routing the behaviour, and a harness that can score it

On the branch `dual_tracking` (not merged to main), `AgentConfig.use_router` is widened: the track now drives the clarification policy (buying -> `FixedPolicy`, browsing -> `InfoGainPolicy`), per-track rerank weights, an optional buying-track hard filter, and per-track recommendation timing, with `detect_turn_intent` re-checking the track every turn. `use_router=False` restores the flat pipeline bit-for-bit.

Why it stayed on a branch: on the fully-cooperative public simulator this costs ~0.013 (public 0.9305 -> 0.9177, dev 0.9418 -> 0.9268, holdout 0.9136 -> 0.9041, one adversarial bucket -0.013) and gains nothing, because that simulator hands over every constraint on the broad "anything else?" question regardless of track.

`tools/stress_harness.py --customer browse-gated` makes the browsing customer realistic — it discloses only when asked a pointed question — and there routing lifts browsing Hit@10 0.59 -> 0.95 and MRR 0.24 -> 0.67 (+0.147 overall) with buyers unchanged; misroute cost is ~10× asymmetric (browser-as-buyer -0.66 MRR vs buyer-as-browser -0.07). The `stress_harness` branch also composes that with paraphrase, and its retrieval diagnostic shows the real gap: under a gated + paraphrasing browser, **9/80 browsing targets never enter the pool** vs 1/80 for buyers — the retrieval weakness is on the browsing track. Write-ups:

`docs/team/dual_track_routing.md`, `docs/team/stress_harness.md`.

Ideas for this stage

- ~**Route the *question*, not the retrieval.~ Done on branch `dual_tracking`** (`starter/agent.py _policy_for / _track`): buying keeps the broad question, browsing runs `InfoGainPolicy`. Measured net-negative on the public simulator, strongly positive on `tools/stress_harness.py --customer browse-gated`.

- ****Route the *retrieval* for browsers (not buyers).** The stress_harness diagnostic shows a gated + paraphrasing browser loses 9/80 targets out of the 300-pool (buyers: 1/80). A browsing-only query-expansion route, or a category-only fallback when disclosure is too sparse to form a query, is the next lever — and it is the *opposite* of "narrow the buyer's pool" (buyers are the healthy case; rerank.py:depth=300 exists because targets already sit deep in BM25 order).
- **Detect scenario, not just track.** The router distinguishes two of four scenarios. Detecting *boundary* customers (people who answer "I have no preference") early would let the agent stop spending questions on someone who won't answer them — currently that is only learned after a wasted turn, via dead_attributes in src/state.py. The branch confirms the cost of *not* doing this: boundary customers open identically to browsers, get routed to the browsing policy, and lose ~0.24 MRR on the public dev split.
- **~Confidence-weighted routing.** Partially addressed on branch dual_tracking: classify() still returns a hard label, but _track re-evaluates every turn via detect_turn_intent, so a wrong turn-1 call is corrected once the customer discloses (browsing -> buying, one-way).

S3 — Dialog state

File: src/state.py

This is the stage that carries the score. Everything else in this document is a refinement on top of it.

What this stage does

Remembers the conversation: what the customer has said, when they said it, what they have declined to answer, and whether they changed their mind.

Why it was needed

The baseline was stateless. Recall consequence #3 from §3: the customer discloses information only in response to a question, and the baseline never asked one. So the baseline was solving a much harder problem than the one posed — identifying one product in 50,000 from a single vague sentence like "I'm looking for Watches Wrist Watches".

Here is what the agent actually receives across a session once it starts asking:

```
turn 1 "I'm looking for Watches Wrist Watches. Stainless Steel Band"
turn 2 "For that, what matters is: Water Resistant; 3 Year Battery."
turn 3 "For that, what matters is: Day / Date Indicator; Stainless Steel Band."
```

Every session hides **exactly four** constraints, disclosed up to two per answered question. Accumulating them across turns instead of discarding them is the difference between describing "a watch" and describing "*this* watch".

What changed

DialogState stores every message with **provenance** — the turn it arrived on and a weight. It exposes two views of the same history:

- full_text() — everything the customer has said. Maximum recall.
- focused_text() — only the currently authoritative turns. Maximum precision.

Until the customer changes their mind, these are identical.

Intent override — a deliberate deviation from the brief. The problem statement calls for "slot erasure and rewriting": when the customer reverses a preference, delete the old one. We **down-weight instead of erasing**, to PRE_OVERRIDE_WEIGHT = 0.35 (src/state.py:38).

The reason is specific to how this evaluator constructs an override. Looking at behavior_for() (evaluator/local_evaluator.py:78-86), the "discarded" preference is *still drawn from the target product's own metadata*. Erasing it therefore throws away accurate information about the very item we are trying to find, and costs

score. Down-weighting keeps the evidence usable while letting the post-override turns dominate, and `focused_text()` gives retrieval a clean post-override view in case the private simulator uses genuine decoys instead.

This deviation is flagged here, in `README.md`, and in the module docstring, because a reader comparing the code to the brief will otherwise think it is a bug.

Two other signals the state tracks, both read by the clarification policy:

- `dead_attributes` — when the customer says "I don't have a preference for material", that attribute is recorded as dead so the agent never asks again. Re-asking a declined question wastes a turn and reads as not listening.
- `productive_turns` — whether an answer actually disclosed something new. This is how the policy tells the difference between "the customer is still telling me things" and "I have exhausted this line of questioning".

Reading a message that was not written to a template. Both signals above are computed from `constraint_spans()` (`src/text.py`), which splits a customer message on punctuation and keeps each resulting fragment as a stated constraint. That is exactly right for the evaluator's own wording, where a colon always separates the framing from the value — "For that, what matters is: Stainless Steel Band." splits cleanly into the value alone. It is wrong for any customer who phrases things freely, and two such cases were found and fixed:

- **Carrier framing was being glued onto the value.** "I'd also want it to be synthetic sole." has no separator between the framing and the value, so the whole sentence was recorded as one constraint — `i d also want it to be synthetic sole` — and the real value, `synthetic sole`, was lost. "One more thing - a breathable net weave." recorded a bogus `one more thing` alongside the real value. The fix strips a run of stopwords off *both ends* of each fragment, generalising the leading-only strip `pair_spans()` in the same file already did. This matters well beyond this stage: `query_spans()` feeds S6's span-coverage signal, and a glued fragment almost never appears literally in a product's text, so the pollution was quietly weakening the main ranking signal rather than merely making the ledger untidy.
- **A refusal was being recorded as a disclosure.** A customer who is only browsing may answer a broad question with a stall — "I'm still just browsing, ask me about one particular thing" — which split into two invented constraints *and* counted as a productive turn, resetting the unproductive streak. Since that streak is the only input to stagnation detection, a conversation stuck in exactly the loop stagnation recovery exists for could never trigger it. A `STALL_CUES` pattern now treats a stall like a decline for recording purposes, but **not** for `dead_attributes` — a stall is not "no preference for X", so the attribute stays live. The check is deliberately skipped on turn 1, because the evaluator opens every browsing session with "I'm looking for {category}, but I'm still exploring." — matching the pattern. Applying it there would discard the single most informative message of every browsing session.

Both were found by auditing this branch against a sibling branch that had already fixed them, and both were confirmed present here by running them, not by reading the code.

Measured effect

0.1067 -> 0.7811. Roughly 90% of the total improvement in this project comes from this stage plus the decision to ask a question at all.

Ideas for this stage

All four override ideas below were investigated together and closed as not worth pursuing. The reason is one measured fact: `behavior_for()` draws both `old_value` and `new_value` from the *same target product's* intent card, and across all 46 override sessions in the public and hard sets, **not one replaces an exclusive facet value with a different one** — 25/30 public overrides are cross-slot ("Buckle closure" -> "leather"), 4/30 are feature -> feature ("Stainless Steel Band" -> "Water Resistant", both true of the target), and the single material -> material case repeats the same value. The override is an *emphasis shift*, not a retraction. Full measurements in `docs/team/rerank_signals.md` §6-§8.

- **~~Tune the override weight.~~ Closed.** `PRE_OVERRIDE_WEIGHT = 0.35` is inert *and* has nothing to express: there is no discarded constraint to down-weight, so no value of the constant is more correct than another.
 - **~~Per-constraint provenance, not per-turn.~~ Closed.** Built as a four-variant ablation over the conflict-scoring path (turn-1 exclusion × full-vs-focused history); every variant scored flat or worse than the shipped agent. Provenance is real — `focused_text()` in the conflict path turns out to be nothing but a turn-1 filter — but acting on it more precisely loses score, because the turn-1 category framing is a genuine constraint.
 - **~~**Detect *partial* overrides.~~ Closed.**** A partial override is the *only* kind this simulator produces, and the correct handling of it is to add the new constraint without touching the old one — which is already what the agent does.
 - **~~Contradiction detection.~~ Closed for this evaluator.** `customer_reply()` only ever *adds* constraints, all drawn from one target's card, so the customer can never state two incompatible ones. Still a real product behaviour; not a scoring lever here.
-

S4 — Clarification policy

File: `src/policy.py`

What this stage does

Decides **what to ask about next** — the value placed in `ask_attribute`. Legal values are category, material, color, size, style, brand, budget, feature, use_case, other.

Why it was needed

By consequence #3 in §3, this field is the agent's only means of learning anything. The baseline left it `null` permanently.

What changed — two implementations

`FixedPolicy` (`src/policy.py:40`) — **what ships**. Asks the broadest available question every turn, falling through a ladder (feature, use_case, style, ...) as attributes are declined. Simple and, as measured below, effective.

`InfoGainPolicy` (`src/policy.py:72`) — **the more interesting design**. Chooses the question that would most reduce uncertainty about which candidate is correct.

The intuition, using colour. Suppose the shortlist holds 300 candidates: 150 black, 150 white. Asking "what colour?" eliminates about half whatever the answer is — a genuinely useful question. Now suppose 299 are black and 1 is white. The same question almost certainly returns "black" and eliminates almost nothing. **The value of a question is how evenly it splits what you're unsure about.** The formal name for that measure is **entropy**.

The policy scores each attribute as:

$$\text{value(attribute)} = \text{coverage} \times \text{gain_ratio} \times \text{answerability}$$

Each term earns its place, and two of them exist because the naive version failed:

- **coverage** — the fraction of candidates for which the attribute is even resolvable. This is what stops the agent asking about budget: **78.9% of this catalog has** `price: null`, so a budget question is unanswerable most of the time.
- **gain_ratio** — entropy divided by its maximum possible value. *This was a bug fix.* Raw entropy is biased toward attributes with many distinct values. Measured on a real session, brand scored **6.1 bits** against colour's **2.3** purely because the catalog holds thousands of distinct stores, so the agent opened every conversation by asking which brand the customer wanted. Normalising by $\log_2(\text{number of distinct values})$ removes the bias. This is a classic trap in information-gain methods and is worth knowing about generally.

- **answerability** — the probability a *shopper* can answer. Even after the gain-ratio fix, brand still won: every product has a brand, so coverage was 100% and the split was near-perfect. But few people browsing for a belt can name the brand they want. This term encodes shopper behaviour, not catalog structure — and the customer's own `preference_tags` nudge it, which is the one place the user profile does real work.

Broad questions ("anything else that matters?") are scored on the same scale rather than special-cased: they can be answered as long as something remains undisclosed, and they return whichever fact the customer thinks is most important, so their value is the average gain across resolvable attributes scaled by how many facts a broad answer tends to surface.

Measured effect — and why the simpler one ships

	dev	holdout
FixedPolicy	0.8738	0.8374
InfoGainPolicy	0.8369	0.8141

InfoGainPolicy actually finds the target *slightly more often* on dev (hit rate 0.958 vs 0.950) — but ranks it considerably worse (MRR 0.703 vs 0.814). The cause is understood: specific questions surface fewer facts per turn than broad ones, and thinner evidence ranks worse even when it still retrieves.

The decision rule was fixed **before** the measurement: ship whatever wins on data that was not tuned against. FixedPolicy ships. InfoGainPolicy stays in the repo, fully documented and selectable:

```
Agent(catalog, AgentConfig(policy=InfoGainPolicy(agent.facets)))
```

Phrasing — pool-aware clarification wording (`src/phrasing.py`)

Separate from *what to ask* is *how to say it*. FixedPolicy keeps `ask_attribute="other"` because that is the score-optimal extraction (§3 consequence: the simulator returns two constraints of any type for other and can return zero for a specific attribute, which also retires that attribute). The simulator never reads the English message, but the two outputs still form a public contract: the natural question now always agrees with the structured field.

`clarify()` (`src/phrasing.py`) builds the message. From turn 2 onward — once the retrieval pool has been shaped by something the shopper actually said — it looks at the live reranked pool and, for each of `material` / `colour` / `style` / `size` / `use_case` that has not been asked or declined, measures how evenly the pool is split on it — the same `gain_ratio` (entropy ÷ maximum entropy) the InfoGainPolicy uses. Facets that are genuinely split (at least 25% of the pool resolves it, the top value holds no more than 90% of the mass, two or more values present) are collected, ordered by split quality, and the one at `turn_count % count` is voiced as optional guidance for an open other question. It names the top two or three values while explicitly allowing the customer to provide another detail:

"The shortlist differs on material: leather and canvas. Is that important, or is there another detail I should prioritize?"

When `ask_attribute` is specific, the grounded path is restricted to that exact facet: a `material` action may name leather and canvas; a `size` action can never voice a material split. Non-voiceable attributes use their matching specific question bank.

The turn-2 gate does **not** require a *productive* turn. Single-word disclosures ("leather", "black") never form a multi-word constraint span, so `productive_turns` can sit at 0 for a whole session that is in fact narrowing well — an earlier stricter gate left those sessions (e.g. `public_0198`) on the broad fallback for all ten turns. The per-facet split tests are the real guard against voicing a facet the pool has not split on. On the public set the grounded path now fires on 98% of `turn->=2` clarifications and in 199 of 200 sessions.

On turn 1, and whenever no facet qualifies, it falls back to a seven-way rotation of the broad question ("Anything else I should keep in mind?"); when a specific ladder rung is asked (FixedPolicy.FALLBACK after other is declined) it uses a three-way rotation of that attribute's own question ("Any must-have features?", "How should this fit?"). The whole path is wrapped so a phrasing bug degrades to a question for the same `ask_attribute`, never an unrelated

broad question or an empty turn. brand and budget and category are excluded from the *voiced* facets — brand has thousands of values, budget is null for 79% of the catalog, and the category facet's values are path fragments ("women", "novelty").

Lead-in and intent. The old code prefixed every sentence with the router's one fixed tone string ("To point you in the right direction: "). Now the prefix is drawn from a bank keyed on `route.name`: browsing sessions get soft framings ("To help me narrow things down, "), buying sessions get decisive ones ("To zero in on the right one, "), and half the bank is empty — so many turns carry no prefix at all, which is how people actually talk. On the turn the customer reverses course (`state.override_turn == turn_count`) the prefix is a distinct acknowledgement ("Okay, switching gears — ", "Got it, let's re-aim — ") and every turn after an override is treated as focused (buying prefixes), because a reversal is a decisive act.

Determinism. Each of the three banks (lead-in, grounded template, broad question) is indexed by `zlib.crc32` of the *opening line* plus the turn number (and, for the grounded template, the *voiced* attribute). Keying on the opening rather than the evaluator's random `session_id` means the wording is fixed per session and reproducible across runs, while still differing turn-to-turn and session-to-session. `natural_questions=False` bypasses all of this through `_legacy_tone` and reproduces the old fixed strings byte-for-byte.

This lives in S4 rather than S9 because it is the customer-facing half of the clarification decision, and it belongs *beside* `InfoGainPolicy` — it reuses the same pool-split measure while preserving the question/action contract.

It is deterministic and template-based on purpose: the facet vocabularies are small (§S1) so the space is enumerable, exactly like the rest of the pipeline. An LLM could later replace the grounded-question builder for fluency with this as its fallback; `ask_attribute` and the score do not move either way.

Measured effect

Exactly zero, by construction — the simulator ignores message. Verified in one process (`tools/sweep.py` rows `natural_off / natural_on`):

	dev	holdout	public (200)	adversarial (96)
<code>natural_questions=False</code>	0.9418	0.9136	0.9305	0.8020
<code>natural_questions=True</code>	0.9418	0.9136	0.9305	0.8020

Per-scenario components are identical on every split. The change buys demo / Presentation / Innovation realism (Pillar II's "structured, proactive clarification prompts"), not score. Default on; `AgentConfig(natural_questions=False)` restores the fixed strings byte-for-byte.

Ideas for this stage

- ****Model expected *yield*, not just split quality.**** This is the identified fix for `InfoGainPolicy` and follows directly from the measurement above. It currently estimates *how much a known answer would narrow the field* but not *how many facts the answer will contain*. Broad questions win here precisely because they return two constraints instead of a fraction of one — and the policy cannot see that. Estimating yield per attribute from observed disclosure sizes, rather than the current fixed `broad_yield` constant, is the single change most likely to make the principled policy overtake the simple one.
- **Ask about the top candidates, not the whole pool.** Entropy is computed over 150 candidates weighted by score. Concentrating on the top ~20 — the ones actually in contention — would ask questions that separate the leaders rather than the field.
- **Two-step lookahead.** The policy is greedy, picking the best single question. Some pairs of questions are worth more together than either is alone.

- **Question phrasing from the candidates.** *Done* — `src/phrasing.py`, see "*Phrasing*" above. For `ask_attribute="other"`, the message can name a useful live-pool split but explicitly remains open to another detail. A specific action only voices that same facet. Deterministic, no model, score unchanged by construction. An LLM polish layer would slot in as `_grounded`'s replacement.
-

S5 — Retrieval

File: `src/retrieval.py`

What this stage does

Produces the shortlist of ~300 plausible candidates from 50,000, given everything the customer has said.

Why it was needed

The baseline ran a single query over the current message only. With state (§S3) the query can now be built from the whole conversation.

What changed

Two **routes** — independent searches — whose results are combined:

- 1 **Terms route** — all the customer's words, over the whole conversation. High recall.
- 2 **Focused route** — the same, but over post-override turns only. Runs only when the customer has changed their mind, and gives the reversal a clean channel.

Combining with Reciprocal Rank Fusion (RRF). Two searches return two ranked lists with scores on incompatible scales — one might score 34.4, the other 18.7, and those numbers mean different things. RRF sidesteps calibration by **ignoring the scores and using only the positions**:

$$\text{combined_score}(\text{product}) = \text{sum over routes of } \text{weight} / (60 + \text{position})$$

A product at position 1 contributes $1/61$; at position 10, $1/70$. Appearing high in both lists beats appearing very high in one. Because it needs no calibration, adding a route later cannot destabilise the existing ones — and if a route returns nothing, it simply contributes nothing.

A coarse-category pool route. The routes above all treat the customer's stated category as a *bag of words*. "I'm looking for Jewelry Necklaces" contributes the tokens `jewelry` and `necklaces` to an OR query, where they compete with every other word in the conversation. A random product with "necklace" in its title therefore competes on equal footing with a genuine member of the `Jewelry Necklaces` category — and can beat it, because BM25 rewards a short field containing the word over a long one that merely belongs to the category.

The evaluator gives us something much stronger than words. Every opening message is built as `coarse_category(target's own categories)` (`evaluator/local_evaluator.py`, `initial_message`), where `coarse_category` keeps the two most specific levels of the category path. That makes the stated string a deterministic function of the target's category — and, crucially, it **inverts**. Compute the same function over all 50,000 products and you get 1,115 buckets; look the stated string up as a key and you have a candidate set the target is guaranteed to be in.

Verified on the public set using only the opening message, with no access to the target's metadata: the stated string is an exact bucket key **200/200 times**, the target is inside that bucket **200/200 times**, and the median bucket holds **182 products** — of 50,000.

The bucket is fused in as an ordinary weighted RRF route, its members ordered by popularity (targets are drawn with a popularity-weighted sampler, so the head of a bucket is where they concentrate). Anything the `pool_size` truncation then cuts is appended back, because the point of the route is the *guarantee*, and a truncation that drops the target gives it away.

One trap is worth recording, because the first implementation fell into it and looked fine. Appended members were given a score *below zero* to keep them out of the way of the fused head. But the reranker mixes the retrieval score into its own as $\text{retrieval_weight} \times (\text{retrieval_score} / \text{top_score})$, so a negative filler is not a polite demotion — at a typical RRF top score of 0.03 it is a penalty of about -33, which no amount of span evidence can climb back from. Recall measured a healthy 100% and the score moved by **exactly zero** on both splits. A gate passing is not the same as a mechanism working.

Measured effect

The terms route is **excellent at its job** — **once the customer has told you everything**. Measured over 80 sampled sessions with all four constraints disclosed:

- target present in the shortlist: **80 / 80**
- median position of the target: **1**
- target within the top 10: **69 / 80**

That result set the direction for a long time: **retrieval was not the bottleneck** — **ranking was**, so effort moved to §S6 and a planned dense-vector route was cancelled as unnecessary.

It was true, and it was measured at the wrong moment. Those 80 sessions were scored with *all four constraints already disclosed* — the end of a conversation. Re-measured at **turn 1**, when the customer has said one sentence and the agent must already make its first guess, the same retrieval looks completely different:

turn-1 candidate pool	size	target present	target's rank
lexical routes only	300	80.5%	median 51
with the category pool	median 315	100%	—

Only **66%** of those 300 candidates were even in the target's category. Session `public_0001` is the whole problem in one line: the customer says *"I'm looking for Jewelry Necklaces. A key requirement is: Material:alloy."*, and the target is **#11 by popularity inside its own 329-product category** while sitting at rank **113 of 300** in our pool.

Recall was never fixed — it was measured late. Turn-1 recall is a different quantity from end-of-session recall, and with §S7's one-candidate-per-turn sizing the agent is *spending* its first guess exactly where recall is worst.

Effect of the category pool route:

set	before	after
public (200)	0.923487	0.934554
hard (96)	0.793780	0.826035
generated (200)	0.9104	0.9183
dev / holdout	0.9292 / 0.9149	0.9417 / 0.9239
paraphrase:heavy+browse-gated	0.770651	0.874730

Hit@10 on public stays at 1.000; the public gain is MRR 0.8810 -> 0.9082 and MTTC 3.040 -> 2.895. The result that matters most is on the *other* sets, which do not share the public generator's sampling: Hit@10 rises on hard (0.885 -> 0.927), on generated (0.990 -> 1.000) and under stress (0.880 -> 0.990). `tools/observe.py` on the hard set shows `never_retrieved` going **6 -> 0**, with two of those six landing in `ranked_out` (5 -> 7) — they are now in the pool every turn and §S6 fails to surface them, which is a ranking problem this route deliberately does not touch.

The route weight is 1.0, and it was **not** chosen on the public splits. 1.5 is the argmax of both dev and holdout and the hard set rejects it by 0.015 — the same shape as the rejected popularity sweep in §6. 0.7 and 1.0 are the plateau where all four sets agree.

Finally, the honest limit. This inverts a function the *evaluator* uses to write its own prompts, so part of its public-set strength is a property of the session generator rather than of our search. A customer who renames the category never produces an exact bucket key, and `tools/stress_harness.py` cannot test that — it rewords the constraint and keeps "I'm looking for {category}" verbatim. Perturbing the stated category directly:

stated category	target still in pool
verbatim	100.0%
lowercased, & -> and	100.0%
word order shuffled	100.0%
singularised	63.5% -> 100.0%
last word dropped	88.5% -> 81.0%

Singularisation failed at 63.5% for a dull reason — "Necklaces" and "Necklace" are different tokens — so the fallback vocabulary now indexes a naive singular form of each key token, trading the word-drop case down to 81%. When nothing overlaps, `match_pool` returns nothing and the route contributes nothing, which is the previous behaviour exactly.

Ideas for this stage

- **Tune `weight_focused`.** Set to 0.8 by judgement (`src/retrieval.py:36`) and never swept. Like `PRE_OVERRIDE_WEIGHT`, it sits directly on the weakest scenario.
- **Weight words by recency.** Every word in the conversation currently counts equally, whether said on turn 1 or turn 5. Later disclosures are more specific and arguably deserve more weight — a cheap experiment on top of the existing `utterance.weight` machinery.
- **~~A category route.~~ Done — see "A coarse-category pool route" above.** The parenthetical ("the same idea may work better as a ranking signal") turned out to be the trap, not the shortcut: the ranking-signal version already existed as `category_weight` and `tail_weight` (§S6) and could not help, because a ranking signal reorders a pool and cannot recall a product retrieval never returned. Both are still worth keeping — ablating either is flat-to-negative, since the fused pool is not category-pure — but the recall half had to be a retrieval route. Worth +0.011 public and +0.032 hard.
- **Give the pool route a stemmer.** The paraphrase fallback matches raw tokens plus a naive singular form (strip a trailing s). That covers plural/singular and nothing else — no British/American spellings, no synonyms ("jewellery", "pendant"). A real stemmer would extend the guarantee to customers who rename the category. (Scoring bucket keys with the `src/embed.py` encoder was the other candidate; that encoder has since been deleted — see "The two optional layers" below — so a stemmer or an explicit synonym table is what is left, and either is `stdlib`.) This is the single biggest robustness gap the route leaves.
- **Teach `tools/stress_harness.py` to reword the category.** It paraphrases the constraint and keeps "I'm looking for {category}" verbatim, so it cannot measure the gap above — the stress score for the pool route is optimistic by construction and the perturbation table had to stand in for it. A category-drift stressor would make that number real.
- **~~Dense retrieval, if recall ever becomes the bottleneck.~~ Built, measured, removed.** A local sentence-embedding route catches paraphrases that share no words, and it was built (`src/embed.py`, `RetrievalConfig.use_dense`) after this bullet was first written. It recovered **none** of the `never_retrieved` tail it was aimed at and was slightly negative overall (`docs/team/dense_route.md`, `docs/team/branch_state_encoder_eval_changes.md` §3d), so change 20 deleted it rather than keep it as an off-by-default flag. If the private sessions paraphrase heavily enough for recall to fall, the idea becomes relevant again — but it starts from the measurements above, not from a knob.

S6 — Reranking

File: src/rerank.py

The second-largest win in the project, and the clearest illustration of the retrieval-versus-ranking distinction from §2.

What this stage does

Takes the ~300 candidates and reorders them so the best is first.

Why it was needed

From §5: the target was already in the shortlist 80/80 times at median position 1, but only reached the top 10 in 69 of 80. The gap was pure ordering. And ordering is worth a lot — MRR carries 30% of the score, and by consequence #1 in §3 you only ever get one chance at it.

What changed

The insight. The constraints the customer discloses are not paraphrases. They are **copied verbatim from the target product's own metadata** — `intent_card()` (`evaluator/local_evaluator.py:52`) builds them directly out of the product's features and details fields. So when a customer says "stainless steel band", those exact words are sitting in the target product's text.

That makes an unusually strong signal available: check whether each candidate's text *literally contains* the phrases the customer used.

```
customer said: "stainless steel band", "day date indicator"

candidate A text: "...analog watch, stainless steel band, day date indicator..." -> 2 matches
candidate B text: "...leather strap chronograph..." -> 0 matches
```

The final score for each candidate combines three signals:

```
score = span_coverage          (dominant - verbatim phrase matches)
       + normalised_retrieval_score (BM25's opinion, scaled to 0..1)
       + 0.4 x popularity      (the tie-break - raised from 0.02, see below)
```

Longer phrases count slightly more (`length_bonus`), because a five-word match is rarer than a two-word one.

Popularity was deliberately near-zero — and then measurement moved it. For most of this project `average_rating` and `rating_number` were held at weight 0.02: the target is *one specific person's purchase*, not a bestseller, so a strong popularity prior would drag the ranking toward famous products. That reasoning is right in general and was wrong about the margin: change 12 dissected every session where the target sat at rank 2-10 behind an impostor and found all lexical signals *exactly tied*, the retrieval score picking the impostor 33/33 (BM25 rates the same matched words higher in a thinner listing), and popularity picking the target 31/33 — because a product someone really bought tends to be a reviewed, documented product. At 0.02 the signal that was right 94% of the time was drowned 50:1 by one that was wrong 100% of the time in exactly the regime that held all the remaining headroom. The weight is now 0.4 — mid-plateau, every split up, measured in `docs/team/rerank_signals.md` §10. The general lesson survives in refined form: popularity is a bad *primary* signal here (the coordinate-ascent argmax pushed it to 0.8 and regressed the adversarial set), but the *tie-break* is worth far more than a token weight.

Why this lives in ranking and not retrieval. Verbatim matching was *first* built as a third retrieval route, using FTS5 phrase queries. It scored **0.6859 against the floor's 0.7799 — a large loss**. Investigation showed why: as a search route it found the target in only **47 of 80** sessions, against the terms route's 80/80. Fused into retrieval, it injected 33 sessions' worth of noise ahead of good results.

The same evidence, applied to a shortlist the terms route had already filled correctly, is pure gain.

Precision-oriented signals belong in ranking; recall-oriented ones belong in retrieval. This is the concrete example promised in §2.

****Later addition — matching the category tail.**** The opening message names the target's coarse category, and the evaluator builds that from the two most specific levels of the target's category path (`coarse_category`,

evaluator/local_evaluator.py:126): a target filed under Novelty > Women produces "I'm looking for Novelty Women".

Rewarding candidates that share *ancestors* with the opening cannot use this. Consider two candidates when the customer says "Novelty Women":

target	... > Novelty > Women	tail = "Novelty Women"	named
candidate	... > Novelty > Women > Tops & Tees > T-Shirts	tail = "Tops & Tees T-Shirts"	not named

The candidate shares *every* ancestor the target has, so ancestor overlap scores it just as highly — yet its own two most specific levels were never mentioned. Scoring the tail separates them: award a point for each of the candidate's two deepest category levels whose words all appear in the opening. Matching is by token containment rather than by parsing the opening's sentence pattern, so a paraphrased private-set opening still works.

In the last remaining public-set miss (public_0020) this cut a **159-way tie** down to the few candidates on the right leaf. It is worth **+0.0058** on the public set and **+0.0053** on the adversarial set, entirely through better ordering — hit rate does not move on either set, which is exactly what a reranking signal should look like (RerankConfig.tail_weight = 0.8).

Later addition — opt-in LLM semantic reranking (DeepSeek), gated on pool ambiguity. Every signal above is exact-token: span_coverage, facet agreement and the category tail match all go to zero the moment a candidate says "cowhide" where the customer said "leather". A real language model can read past that. src/llm.py adds a small client (urllib only, no new dependency) for DeepSeek's chat API: once per turn it can be shown the top handful of already-ranked candidates and asked to reorder them by how well each matches the conversation so far.

Two design choices keep this from ever costing the guaranteed offline score. First, it is *fused*, not a replacement: the model's suggested order becomes one more additive term in rerank()'s scoring formula

(RerankConfig.llm_weight, the same shape the removed dense_weight term had), so a bad reorder from the model can only nudge the ranking, never override the lexical evidence outright. Second, it is *gated*:

RerankConfig.llm_gate_margin reads state.leader_margin — the previous turn's gap between the top two candidates — and only calls the model when that gap is small (< 0.05, the same pool-shape signal the removed dense_gate_over_general used). A pool with a clear lexical leader has nothing to gain from a second, nondeterministic opinion; an undecided one is exactly where a semantic read can break a tie exact-token matching cannot see. RerankConfig.llm_weight = 0.0 by default, so nothing about the shipped agent's offline guarantee changes — every existing test and config still runs with zero network calls.

LLMReranker.rank() treats *any* failure — no API key, a timeout, a malformed reply, an id the model invented — as "no opinion" and returns None; rerank() then leaves the lexical order exactly as it was. There is no path where a flaky network call can make a turn score worse than leaving the layer off.

Measured effect

0.7799 -> 0.8543 for verbatim span coverage — the single largest gain after dialog state. The signals added since (facet agreement, category agreement, and the category tail match) are recorded per change in agent_changes.md.

The LLM layer, measured with live DeepSeek calls (change 17, agent_changes.md), against this branch's current baseline (public 0.923487) — on the codebase with change 16's three lexical-span bug fixes in place:

split	offline baseline	with the LLM layer on, gated	Δ
public set (200, official)	0.923487	0.9254	+0.0019
holdout (80)	0.9149	0.9218	+0.0069
stress: paraphrase:heavy+browse-gated	0.77065	0.77432	+0.0037

hit@10 never regresses anywhere — the layer only ever moves ordering. It stays off by default

(RerankConfig.llm_weight = 0.0): the gains are real but small (inside or just past this project's own noise floor), and one scenario — intent_override under stress — gets measurably worse (MRR 0.8028 -> 0.7750), a genuine

trade-off rather than something one run settles. An earlier measurement, taken before change 16's fixes landed, found roughly double this stress-harness gain (+0.0071) against a baseline the fixes have since moved (0.76086 -> 0.77065) — the same lexical-signal repair that shrank the branch's dense embedding route's stress gain to 16% of its former size shrank this layer's gain too, by about half, for the identical reason: both were partly compensating for `query_spans()` carrying glued, unmatched carrier text. See `agent_changes.md` change 17 for the full per-scenario breakdown.

Ideas for this stage

- **~~Facet agreement — the clearest unused signal in the repo.~~ Done.** `src/facets.py` was extracting material, colour, size, brand and price band for every product while no ranking signal read any of it. It now feeds `RerankConfig.facet_weight` (PR #3).
- **~~Category anchoring.~~ Done, twice.** The motivating failure was a target belt whose customer had disclosed only "buckle closure" and "100 leather" — both generic — leaving the target at **rank 15** behind a dozen other leather belts. This is now covered by `category_weight` (PR #4) and, more sharply, by the category tail match described above. Together they closed the last public-set miss.
- **~~Learn the signal weights.~~ Done — with a twist** (change 12). `tools/fit_weights.py` fits all seven non-definitional weights by coordinate ascent directly on the technical score (Metzler & Croft 2007), dev split only, standard library, offline. The sealed holdout confirmed the fit's direction (+0.019), but the dev argmax regressed the adversarial set, so what shipped is the single change the fit and the near-miss anatomy both pointed at: `popularity_weight` 0.02 -> 0.4. Public 0.9199 -> 0.9305, every split up. The honest lesson: the fit finds the direction; the gates decide the magnitude.
- **~~A negative-evidence signal.~~ Done.** Nothing used to penalise a candidate that *contradicts* a stated constraint — a customer who said "grey" and a candidate whose text never mentions grey was merely left unrewarded, not pushed down. This ships as `RerankConfig.facet_conflict_weight` (`_facet_conflicts`), judged against `focused_text()` and guarded so that silence is never punished.
- **~~LLM semantic reranking (Tier 2 #3, docs/team/ideas_to_integrate_llm.md).~~ Built, measured, kept — off by default.** Unlike the cross-encoder (change 11, removed), a real DeepSeek call fused into the top of the pool and gated on `state.leader_margin` moves every split flat-to-positive with `hit@10` never regressing — but the gain is small and one scenario (`intent_override` under stress) regresses, so `RerankConfig.llm_weight` ships at 0.0 rather than flipping the default. See the measured-effect table above and change 17.
- **Correct the retrieval score's length bias at its source.** The near-miss anatomy says the impostor wins on the retrieval score alone, and the reason is BM25's length normalisation favouring thin listings. Three attempts to correct it *after* the fact — as an additive document-length prior — all failed the adversarial gate (see §6). The untried route is to recompute a length-corrected BM25 over the 300-candidate pool inside the reranker, choosing the normalisation strength directly: SQLite's `bm25()` fixes it, and the fused score the reranker sees is built from BM25 *ranks*, which have already discarded the magnitudes any correction needs. ~30 lines, still standard library.
- **What was already tried and rejected here:** weighting phrases by how rare they are within the shortlist. It moved dev by 0.0002 and holdout not at all — a pool retrieved by those same words has little rarity spread left to exploit. The code was deleted rather than kept as a dead option; see §6. Also rejected: the document-length tie-break in three forms, and category-path precision, which one session made irresistible and 37 killed (§6).

S7 — Recommendation timing

File: `starter/agent.py` (`_shortlist`, `_confident`)

What this stage does

Decides whether to show a list this turn, or stay quiet and ask another question.

Why it was needed

This stage exists entirely because of consequence #1 in §3: **the session ends at the first hit, freezing the rank forever**. Showing an uncertain list early risks catching the target at position 9 and locking in $RR = 0.11$, when waiting two more turns might have ranked it first.

The marginal-value table in §3 quantifies the trade: a rank improvement is worth $\sim 13\times$ the turn it costs. So the agent should be patient.

What changed

Hold until turn 3 — since superseded. For a long time the agent showed nothing on turns 1-2, when the customer has disclosed little. Measured on holdout at the time:

First recommendation on	Score
turn 2	0.8088
turn 3	0.8349
turn 4	0.8196

Turn 3 won on both splits: turn 2 was too eager, turn 4 wasted turns after the evidence had arrived. That table was measured while the first slate was four candidates wide, and the "too eager" penalty is precisely the cost of banking a bad rank early. Once the slate narrowed to a *single* candidate (see "Sniper sizing" below) there is no bad rank left to bank, and the conclusion inverts — `first_recommend_turn` now ships at 1. The table is kept because it is the evidence that the emit turn and the slate width are one decision, not two.

Confidence gating. A fixed turn number is crude — sometimes the answer is obvious on turn 2. `_confident()` allows early recommendation when the top candidate clearly leads:

```
(best_score - runner_up_score) / best_score >= 0.20
```

Margin	holdout
0.0 (disabled)	0.8349
0.15	0.8391
0.30	0.8389
0.50	0.8371

Every enabled value beats disabled, on both splits, and **MRR is unchanged while MTTC improves** — exactly the intended mechanism. The agent gets faster without ranking worse.

The default is 0.20, not 0.15. 0.15 scored highest, but the curve from 0.15 to 0.50 is flat, which means the differences between them are noise. Picking a split's argmax is how you buy noise. Mid-plateau is the defensible choice — the same reasoning applies to `broad_yield` in `src/policy.py`, whose dev results (0.8530 at 5.0, 0.8651 at 8.0, 0.8634 at 12.0) are one plateau.

Narrow the first slate. `list_size_ramp` says how many candidates each turn reveals; the last entry applies to every later turn. It shipped flat at (10,) for a long time, then (4, 10) — four candidates on turn 3, ten from turn 4. It now ships at (1, 1, 1, 1, 10); this paragraph explains the (4, 10) step, and "Sniper sizing" below explains the rest of the way.

Showing *fewer* products scores better, which sounds backwards until you line up three facts about the evaluator. The session ends the instant the target appears in a shown list. The rank it held at that instant becomes the reciprocal rank, permanently. And a list that misses costs nothing but a turn.

So every list is a bet. Reveal ten on turn 3 and you maximise the chance of converting *now* — at whatever rank the target happens to hold. If it is sitting at position 7, you have just banked $RR = 1/7 = 0.14$ and the session is over. Reveal four, and that same target stays hidden; you spend a turn, the customer discloses another constraint, the reranker re-scores with it, and the target typically surfaces near the top. Now you bank 1/1 or 1/2.

The elimination scan is what makes this safe. Products already shown are excluded from later lists, so the six held back on turn 3 are simply the top of the survivor list on turn 4 — nothing is skipped, the same walk is just taken in smaller steps.

First slate	dev	holdout	generated	hard
10 (flat)	0.9233	0.9048	0.9181	0.7944
3	0.9254	0.9146	0.9212	0.7968
4	0.9268	0.9096	0.9197	0.7981
5	0.9295	0.9100	0.9210	0.8001

Three, four and five all beat the flat ramp on all four sets — that is a plateau, so 4 ships as its midpoint rather than 5, which happens to top two columns. The decision rule was fixed before 4 was measured, which is the point: choosing the winner after seeing the table is how you buy noise, exactly as with the 0.20 margin above.

Narrowing a *second* turn seemed not to be more of the same good thing. (5, 5, 10) scores 0.9272 / 0.9044 / 0.9187 / 0.7934, dropping holdout and hard below the flat floor. Each session holds four constraints and the customer discloses at most two per turn, so by turn 4-5 no further evidence is coming and waiting longer spends turns without buying rank. That reasoning is about *evidence*, and about evidence it is correct — which is exactly why it pointed the wrong way, as the next section explains.

Sniper sizing: one candidate per turn. The argument above treats a narrow slate as a way of *deferring* commitment until better evidence arrives. Read that way, narrowing past the point where evidence stops arriving is pointless, and (5, 5, 10) is the proof.

But deferral is not the only thing narrowing buys, and at a width of one it is not even the main thing. Count the ranks a slate can produce. A slate of ten can land the target on any of ten positions, worth anywhere from $RR = 1.0$ down to $RR = 0.1$. A slate of five can land it on five. **A slate of one can only ever land it on rank 1.** That is not a bet that pays off better when the evidence improves — it is a bet that cannot lose on rank at all, whether or not another constraint is ever disclosed.

So the value of narrowing is not smooth in the slate width. It improves gradually from ten down to about four for the deferral reason, and then there is a step at one, where a different mechanism takes over. (5, 5, 10) sits in the flat part of that curve and pays two turns for nothing; (1, 1, 1, 1, 10) reaches the step.

The elimination scan is what keeps this from being a coverage disaster. Products already shown are excluded from later slates, so four singles followed by a ten-wide turn reveal **fourteen distinct products**, not ten — a *deeper* walk than the flat ramp ever took, taken in smaller steps. The usual precision/coverage tension does not apply, because the narrow slates are not throwing candidates away; they are only spreading the same walk across more turns. This is why Hit@10 does not pay for the MRR gain, on any dataset.

And once the slate is one candidate wide, holding it back until turn 3 stops making sense. An early guess that misses costs one turn — 0.02 of score. An early guess that hits banks $RR = 1.0$. So `first_recommend_turn` moves from 3 to 1, and the agent opens by naming its single best guess.

Where to widen back to ten, measured on all four sets:

widen at turn	dev	holdout	generated	hard
pre-sniper (4, 10)	0.9268	0.9096	0.9197	0.7981
5	0.9521	0.9220	0.9322	0.8135
6	0.9469	0.9222	0.9342	0.8131
7	0.9468	0.9257	0.9354	0.8129

Every sniper row beats every pre-sniper row on every set — that is the finding. Between the three, the spread is 0.001 in the mean and each column has a different winner, which is the signature of noise rather than of a real

ordering. 5 ships, but *not* because dev likes it: it is the only one of the three that costs no session on any set, because widening earlier leaves more wide turns as a safety net. Choosing on hit rate rather than on score is the same discipline as the 0.20 margin and the 4-wide slate above — with the difference that here the tie-break is a metric, not a midpoint.

Two neighbouring versions of the idea were measured and rejected, and both are instructive.

src/context_programming.py's STAGNATING phase overrides the ramp with its own ten-wide slate when a session stops producing evidence; carrying the singles through it too (stagnation_slate_size=1) makes every hit rank 1 — dev MRR 0.967 against a dev hit rate of 0.967, arithmetically as good as the metric goes — and still loses 0.0146, because it costs three dev sessions of hit rate to get there. A stalled session is the one case where coverage is worth more than rank. Separately, re-ranking the final turn over the full pool while *ignoring* the exclusion memory — insurance against an unparsed override having wrongly excluded the target — costs 0.0049 on dev, because our scan already un-excludes on a parsed override and the unfiltered top ten is mostly products already shown and disproven.

Track-aware margin. confidence_margin is one number for every session, but the two tracks disclose at different rates: a buying session states one hard requirement on turn 1 and two more on turn 2, so by turn 2 its leader is usually already well separated from the runner-up, while a browsing session is still exploring.

buying_confidence_margin (default 0.08) is a second, lower margin used in _confident() only when the router classified the session as buying:

```
margin = self.config.buying_confidence_margin if is_buying else self.config.confidence_margin
```

This lets buying sessions clear the confidence test a turn earlier without changing anything for browsing sessions, whose margin is untouched.

Measured effect

Confidence gating: 0.8543 -> 0.8592, with MTTC improving from 3.58 to 3.41.

Narrow first slate: public 0.9159 -> 0.9199, adversarial 0.7944 -> 0.7981, dev 0.9233 -> 0.9268, holdout 0.9048 -> 0.9096. The whole gain is MRR bought with MTTC at the ~13x odds \$3 prices — on public, MRR 0.8513 -> 0.8690 ($\times 0.30 = +0.0053$) against MTTC 2.975 -> 3.040 (efficiency -0.0065, $\times 0.20 = -0.0013$), netting the +0.0040 observed. Hit@10 does not move on public (1.000) or hard (0.885): this buys rank, not coverage. One cost worth naming — on the 200-session generated set Hit@10 slips 0.995 -> 0.990, a single session that now runs out of turns.

Sniper sizing (first_recommend_turn 3 -> 1, list_size_ramp (4, 10) -> (1, 1, 1, 1, 10)): public 0.923487 -> 0.940083, adversarial 0.841190 -> 0.852227, hard 0.793780 -> 0.813471, dev 0.9268 -> 0.9521, holdout 0.9096 -> 0.9220, and paraphrase:heavy+browse-gated 0.770651 -> 0.784750. On public the entire gain is MRR — 0.8810 -> 0.9339 ($\times 0.30 = +0.0159$) — with MTTC essentially flat at 3.040 -> 3.005, so unlike the (4, 10) step this one does not even pay the ~13x toll: the turns the singles spend are bought back by guessing from turn 1 instead of turn 3. **Hit@10 moves on no dataset** (1.000 / 1.000 / 0.990 / 0.885 before and after), and tools/observe.py reports an identical failure-mode split on the hard set either way — six never_retrieved, five ranked_out. Nothing is retrieved better and nothing is ranked better; what changed is only how much of the existing ranking the scoring rule lets us keep.

Track-aware margin (PR #7, buying_confidence_margin = 0.08): public 0.930502 -> 0.931302, adversarial 0.801978 -> 0.802811, dev 0.9418 -> 0.9428, holdout 0.9136 -> 0.9141. MRR does not move on any split — the gain is entirely MTTC, exactly the confidence-gating mechanism above, narrowed to buying-track sessions. Small relative to the first-slate change because it only moves the turn-2 gate, and only for buying sessions that were already going to clear the margin by turn 3 anyway; see docs/team/agent_changes.md change 14 for the ablation that isolates it from the src/context_programming.py module shipped alongside it.

Ideas for this stage

- **A calibrated stopping rule.** The margin test is a heuristic on raw score gaps. Converting the reranker's output into an actual probability that the top candidate is correct — fitted on the public sessions — would let the agent stop on a meaningful threshold ("80% confident") instead of an arbitrary ratio.
- **Spend the remaining turns.** The agent stops at the first hit, but it does not know that a hit occurred. On sessions heading for a miss it still shows the same list every turn from turn 3 onward. Detecting stagnation — the shortlist stopped changing, the customer stopped disclosing — and switching to a different strategy would attack the 6% of sessions that currently miss entirely. Partly exercised since: the STAGNATING phase now keeps its wide ten-candidate slate while the rest of the session runs on one-wide sniper slates, and that asymmetry is itself measured — forcing the singles through stagnation costs 0.0146, because a stalled session needs coverage where a live one needs rank.
- **~~Diversify a low-confidence list.~~ Tested, and it does not work.** This entry used to argue that spreading the list across distinct categories or brands would raise the chance of catching the target, on the grounds that a miss->hit is worth 2.6x a rank improvement. **That premise is stale:** it was written when Hit@10 was 0.940. Hit@10 on the public set is now 1.000, so there are no misses left to convert and the trade has nothing to buy. An MMR diversity term was implemented and measured anyway: across 260 public slates it brought the target *into* a slate zero times and pushed it *out* 21 times, and Hit@10 moved on no dataset at any setting. The reason is structural — the customer's disclosed constraints are copied verbatim from the target's own metadata, so the crowded top of the list is a cluster formed *around the target*, which sits at its centre rather than being an outlier crowded out of it. docs/team/ideas.md Idea 3 records the full measurement.
- **~~Vary list length with confidence.~~ Done twice — see "Narrow the first slate" and "Sniper sizing" above.** This entry originally guessed that "there is likely no benefit to showing fewer". The opposite is true, and for a reason the guess missed: showing fewer is not about precision, it is about *when you commit*. list_size_ramp went to (4, 10) for +0.0040, and then to (1, 1, 1, 1, 10) for a further +0.0166 once it became clear that the width-one case is a different mechanism rather than more of the same one. The remaining unexercised part is still making the width depend on measured confidence rather than on the turn number — though the headroom is now smaller, since a one-wide slate is already the best rank available and the only question left is when to widen.
- **Better turn-1 ranking is where the remaining headroom is.** With sizing solved, the residual gap to the strongest published submission on this evaluator (0.9748) is MRR 0.934 vs 0.995 and MTTC 3.005 vs 2.19 — both of which are first-guess accuracy, not timing. Their route is a coarse category pool (median 184 products of 50,000, containing the target 200/200 times) plus a strong popularity prior over that pool, which makes the most-popular constraint-matching product the target often enough to win on turn 1. Raising our own popularity_weight reproduces part of that on the public splits and **loses** on both generated sets (docs/team/agent_changes.md change 18), so the honest version of this idea is an S5/S6 problem — a tighter category pool, or a ranker trained on self-play sessions — not a weight to turn up.

S9 — Robustness and tests

Files: starter/agent.py, tests/

What this stage does

Ensures the agent never crashes, always emits a valid response, and that changes cannot silently break the score.

Why it was needed

Consequence #4 in §3: the evaluator swallows exceptions and turns them into missed sessions with no error shown. A crash on turn 1 of every session would produce a score of zero and an entirely clean-looking run.

What changed

respond() **cannot raise**. The whole path is wrapped; any exception degrades that turn to a valid empty response rather than losing the session. Each stage also degrades locally — a malformed search expression returns [], an

illegal attribute is coerced to other, a missing session is recreated rather than raising.

25 tests, in three groups:

File	Covers
tests/test_contract.py	Every response validates against docs/agent_api_contract.json; hostile input (empty, 5000 characters, SQL) never raises; sessions don't leak into each other; top_k respected
tests/test_components.py	Each stage in isolation — including a test asserting the gain-ratio fix, so the brand-cardinality bug cannot silently return
tests/test_regression.py	End-to-end score on the holdout split must stay above 0.78 — the committed floor

The regression test is the important one. The pipeline has many interacting knobs and it is easy to make a local improvement that costs score overall; this is what catches that.

```
python3 -m unittest discover -s tests -t . # 25 tests, ~14s
```

Measured effect

No score gain. It is insurance — and it caught two real bugs during development, including a policy that kept re-asking a question the customer had already declined.

Ideas for this stage

- **Assert the fallbacks never fire.** Defensive layers can mask real bugs. Counting fallback activations and asserting the count is zero on a clean public run would distinguish "robust" from "quietly broken".
- **Property-based testing.** Generating random conversations and asserting invariants (never crashes, never recommends a non-catalog ID, never re-asks a dead attribute) would cover far more input shapes than 25 hand-written cases.
- **A performance regression test.** Latency is currently measured ad hoc. Pinning per-turn latency in a test would catch an accidentally quadratic change before judging day.

The two turn-1 changes together

§S5's category pool and §S7's sniper sizing were built separately and each measured with the other absent, so both tables above understate the shipped agent. Measured together over the same 200 official sessions in one process:

configuration	Hit@10	MRR	MTTC	Score
neither	1.000	0.881	3.04	0.9235
category pool only (§S5)	1.000	0.908	2.90	0.9346
sniper sizing only (§S7)	1.000	0.934	3.00	0.9401
both	1.000	0.961	2.67	0.9550

The gains are +0.0111 and +0.0166 alone, and +0.0315 together — 0.0038 **more** than their sum. That is worth understanding, because the obvious guess is the opposite: both changes act on turn 1, so they ought to compete for the same headroom.

They do not, because they are the two halves of one bet. Sniper sizing stakes the whole turn on a single candidate — it converts a hit into rank 1, but only if that one candidate is the target. The category pool is what decides whether it is: it takes the turn-1 pool from "target present 80.5% of the time at median rank 51" to "present 100% of the time" inside a set that is category-correct by construction. Making the bet bigger and making it more likely to win multiply rather than add. This is also why the §S7 tables, measured over the old retrieval, understate sniper sizing: it was being

asked to bet on a worse candidate.

6. What was tried and rejected

These are first-class results. They are the evidence that the shipped design was **chosen** rather than assumed, and each one is recorded in the relevant module's docstring so the next reader does not repeat the experiment.

Experiment	Result	Outcome
Verbatim matching as a retrieval route	0.6859 vs floor 0.7799	Removed from retrieval, moved to reranking — where it became the second-largest gain. Found the target in only 47/80 sessions as a search route.
Routing pool size and timing by intent	dev unchanged, holdout -0.002	Router scoped down to phrasing only.
Weighting phrases by rarity within the shortlist	+0.0002 dev, 0.0000 holdout	Deleted entirely, not left as an off-by-default flag.
InfoGainPolicy as the default	dev 0.8369 vs 0.8738; holdout 0.8141 vs 0.8374	Kept in the repo and documented, but not the default.
Dense-vector retrieval	not built	Cancelled after measuring retrieval recall at 80/80 — the bottleneck was ranking, and it would have broken the no-network guarantee.
Turn-1 exclusion from facet extraction	public 0.9159 -> 0.9150, holdout 0.9048 -> 0.9035	Rejected. Removes every false conflict against the target (8 public, 5 hard) and still loses — the opening's category words are a real constraint, and the impostors they demote outweigh the targets they cost.
Multi-valued facet extraction	agreement -0.0006 public / -0.0023 hard; conflict-only 0.0000	Rejected. The agreement half dilutes the signal by matching more candidates; the conflict half is exactly neutral and was not shipped, because a code path no measurement justifies does not earn its place.
Explicit constraint ledger (S3)	not built	Cancelled after specifying and measuring all six operations — see below.
Neural cross-encoder reranking (S6b)	dev 0.9268 -> 0.9211, hard 0.7981 -> 0.7944	Built, measured, removed. Loses on every split and every setting; the optimum semantic weight is zero. Code preserved on branch <code>semantic-rerank</code> .
No-span rescore, re-opened after change 12	dev 0.941757 -> 0.941757 (bit-identical), holdout 0.9136 -> 0.9188, hard 0.8020 -> 0.8000	Rejected a second time. Change 12 looked like it should revive it; dev did not move by a single digit, and everything that gained was on the gate split. See below.
Document-length tie-break	dev 0.941757 -> 0.943229, hard 0.801978 -> 0.805064 at $w=0.10$ only	Built in three forms, rejected. The hard-set gate is cleared at one weight with both neighbours failing — an argmax on noise, not a plateau. See below.
The dense embedding term, re-measured after changes 18+19	public -0.0004, holdout +0.0007, hard -0.0046 , paraphrase stress -0.0048	Rejected, then removed. Negative on the two sets built to test the paraphrase claim it existed for. <code>src/embed.py</code> , <code>tools/build_embeddings.py</code> , the <code>dense_*</code> sweep rows and the three third-party pins they needed are all deleted (change 20); the measurements stay here and in <code>docs/team/</code> .
LLM listwise rerank (DeepSeek), re-measured after changes 18+19	public +0.0008 and +0.0017 on two runs of the same config, holdout -0.0005	Rejected for now. Its run-to-run spread is the same size as its effect. Was worth +0.003525 before those changes.

Both optional layers together	public -0.0011	Rejected. Below either alone and below the baseline.
Raising popularity_weight under sniper sizing	dev 0.9521 -> 0.9583, holdout 0.9220 -> 0.9350 at $w=1.0$; generated 0.9322 -> 0.9315, hard 0.8135 -> 0.8072	Rejected. Both public-derived splits improve monotonically and substantially; both <i>generated</i> sets, whose sessions are not drawn from the public set's sampling, are flat to negative, and the hard set falls monotonically to 0.7872 by $w=1.8$. See below.
Singles through the STAGNATING phase	dev 0.9521 -> 0.9322	Rejected. Makes every hit rank 1 (dev MRR 0.967 against a 0.967 hit rate) and still loses 0.0146, because it costs three dev sessions of hit rate. Knob kept as <code>AgentConfig.stagnation_slate_size</code> , defaulting to the wide slate.
Final-turn exclusion bypass	dev 0.9521 -> 0.9472, holdout 0.0000	Rejected. Ported from a rival submission, where it insures against an unparsed override permanently excluding the target. Our elimination scan already un-excludes on a parsed override, so the last turn's unfiltered top ten is mostly products already shown and disproven — it spends the deepest slate of the session on them.

Document length, and the difference between a signal and a shippable signal. The near-miss anatomy asks a narrow question: when the target sits at rank 2-10 behind an impostor, what separates them? On the dev split, 37 sessions, the answer is that every *content* signal is exactly tied and only two things differ — how many reviews the product has, and how long its description is. The target averages 221 tokens, the impostor 104. Length picks the target 33 times out of 37, and it rescues 5 of the 6 sessions where the review count points the wrong way, so it is not the review signal wearing a second hat.

The mechanism is BM25's document-length normalisation. BM25 divides a document's match score by its length, on the reasoning that a long document mentioning "leather" three times is less about leather than a short one mentioning it three times. That reasoning is right in general and wrong here: the products a real customer actually bought tend to be the ones with a filled-out listing, so length correlates with *being the answer*. This is a textbook case of what the IR literature calls pivoted length normalisation (Singhal, Buckley & Mitra, 1996) — the correction exists precisely to close the gap between how likely a document is to be relevant and how likely the scorer is to retrieve it.

Three ways of applying it were built and measured, and none shipped. Adding a term proportional to $\log(\text{length})$ is too small to matter — target and impostor differ by 0.09 on that scale against a retrieval gap of 0.5. Replacing it with the candidate's length *percentile within the retrieved pool* fixes the scale and then overshoots: it gains 0.0006 on dev and costs the adversarial set 0.0068, because the adversarial set deliberately draws thin, sparsely-described targets. Restricting the term to candidates whose content evidence is exactly tied — the regime the anatomy actually measured — is the honest form, and it works on the motivating session (`public_0002`, a dev session, moves the target from rank 9 to rank 3). But across the weight bracket the adversarial set swings non-monotonically by 0.014, roughly one session on a 96-session set, and only $w=0.10$ lands above baseline while 0.08 and 0.12 both fall below it.

That is the whole finding: **a single weight clearing a gate with both its neighbours failing is an argmax on noise, and this project ships plateaus, not argmaxes.** The same rule kept the coordinate-ascent weight vector out of the tree in change 12. The one route not yet tried is to recompute a length-corrected BM25 inside the reranker over the 300-candidate pool, where the normalisation strength is ours to choose — SQLite's built-in `bm25()` fixes it and offers no knob, and the fused retrieval score the reranker currently sees is built from BM25 *ranks*, which have already thrown away the magnitudes the correction needs. `docs/team/rerank_signals.md §11` carries the tables.

The category-path idea that one session made irresistible, and 37 killed. In `public_0002` the target is a men's belt (Men > Accessories > Belts) and the impostors above it are women's golf belts (Sport Specific Clothing

> Golf > Women > Accessories > Belts). The customer said only "Accessories Belts", so the impostors carry three category levels the customer never mentioned and the target carries one — penalising that would separate them instantly, and neither existing category signal can, because both only look at the last two levels and score every one of these candidates identically. Measured across the 37 dev near-misses, it ties on 34. It was never built. One vivid session is a hypothesis, not evidence, and the anatomy is what tells the two apart.

The constraint ledger, and why measuring a design beats arguing about it. A typed Constraint ledger (slot, value, turn, polarity, status) with CARRY / UPDATE / ADD / DELETE / DONTCARE / NEGATE operations was proposed as a replacement for replaying raw utterances. Rather than build it and see, each operation was checked against what the evaluator actually does. Four of the six could not pay: CARRY/ADD is what `full_text()` already does, DONTCARE is already implemented as `Utterance.declined` plus `dead_attributes`, and DELETE/NEGATE cannot fire because `customer_reply()` only ever *adds* constraints.

The sixth, UPDATE, is the interesting one, and it failed for a reason worth writing down. `behavior_for()` draws both the discarded and the new preference from the **same target product's** intent card. Across all 46 override sessions in the public and hard sets, **not one replaces an exclusive facet value with a different one**: 25 of 30 public overrides are cross-slot ("Buckle closure" -> "leather" — a material is added, the buckle is still true), 4 are feature -> feature ("Stainless Steel Band" -> "Water Resistant", both true), and the single material -> material case repeats the same value. What reads as "ignore my earlier preference" is an *emphasis shift*, not a retraction. There is nothing to supersede.

Chasing this also corrected a wrong explanation in the shipped code. `src/rerank.py` judged conflicts against `focused_text()` and attributed that to stale post-override values punishing the target. Measured: single-value extraction over full history picks a contradicted value in **0 of 30** sessions. The guard's real function is dropping turn 1, whose `coarse_category()` framing ("I'm looking for Pants Casual") extracts as a style constraint. Two variants — exclude turn 1 keeping `focused_text()`, and exclude turn 1 using full history — scored **bit-identically on all four splits**, which is the proof. The guard stays; only its comment changed. Full measurements in `docs/team/rerank_signals.md` §6-§8.

The cross-encoder, and what "the model is wrong" actually looks like. Every rerank signal in this agent is lexical — it counts words the customer said that also appear in a product's text. The obvious next move is a *semantic* model that scores how well a product matches a request even when the words differ. `src/semantic.py` does exactly that, with a small neural cross-encoder (cross-encoder/ms-marco-MiniLM-L6-v2, 22.7M parameters, Apache-2.0) that reads a (request, product) pair together and emits one relevance score.

Before building it, we measured the ceiling. An **oracle** reranker — one that cheats by forcing the true target to position 1 whenever it is anywhere in the candidate pool — scores 0.9631 on the public set against the shipped 0.9199, and 0.8823 on the adversarial set against 0.7981. So *any* reranking improvement, neural or not, is playing for +0.043 and +0.084. That is worth having, and it is also the whole prize: 70% of public sessions already rank the target first.

The result was negative on every split and every setting:

variant	dev (120)	hard (96)
off (shipped)	0.9268	0.7981
cross-encoder, weight 0.7	0.9211	0.7944
cross-encoder, weight 0.3	0.9249	0.7959
cross-encoder, depth 20	0.9236	0.7940

Read the weight column downward: 0.7 -> 0.3 -> 0 recovers the baseline monotonically. **When the best setting for a signal's weight is zero, the signal carries no usable information.** There is no threshold to tune toward.

The mechanism is one number. On the 162 turns where the gate fired and the target was in the rescored group, the model moved it **up 46 times and down 74** — mean position 7.63 -> 8.77. It is not slightly miscalibrated; it is pointing

the wrong way more often than the right way. The likely reason is a mismatch between what the model was trained on and what it was asked to do here. Its training data pairs a natural-language question ("how tall is the Eiffel tower") with a prose passage. Here the "question" is simulator boilerplate — *"For that, what matters is: full grain leather; buckle closure"* — and the "passage" is a token-joined blob of title, features and description. On top of that, the job it was given is the hardest discrimination in the pool: separating near-identical products that share every stated attribute, which is precisely where the lexical signals had already saturated.

Cost, for the record: mean turn latency 30.7 ms -> 389.8 ms, p95 73.7 ms -> 1347.8 ms, worst turn 1.48 s, with the gate firing on 28% of rerank calls. docs/competition_specification.md notes that timeouts may count as a miss, so even a *positive* result at this latency would have needed care.

The stage was removed from the working branch and preserved on semantic-rerank: it never ran on the scored path — disabled by default, and a silent no-op without its gitignored weights — so carrying it would have meant carrying a dependency manifest and a download tool for code that never executed. The measurement is the deliverable, and it survives in full in docs/team/rerank_signals.md §9, along with the oracle ceiling it established.

A bug worth learning from. The first version of InfoGainPolicy opened every conversation by asking the customer's preferred *brand*. The cause was using raw entropy: brand has thousands of distinct values in the catalog, so it scored 6.1 bits against colour's 2.3 and always won — even though it is close to useless as a shopping question. This is a well-known failure of information-gain methods, and the fix is standard: divide by the maximum possible entropy to get a **gain ratio**. tests/test_components.py now asserts brand scores below colour, so the bug cannot return.

Raising popularity_weight *under sniper sizing*

Change 12 fitted popularity_weight = 0.4 while the first slate was four candidates wide, where the popularity prior only ever broke ties among candidates that already matched the disclosed constraints. Under a one-candidate slate the prior stops breaking ties and starts *making the decision*, so the value was re-swept from scratch:

popularity_weight	dev (120)	holdout (80)	generated (200)	hard (96)
0.40 (ships)	0.9521	0.9220	0.9322	0.8135
0.70	0.9569	0.9309	0.9328	0.8085
1.00	0.9583	0.9350	0.9315	0.8072
1.40	0.9576	0.9367	0.9298	0.7984
1.80	—	—	—	0.7872

Read the columns, not the rows. Both public-derived splits climb steadily — holdout gains 0.015 at $w = 1.4$, which is a large move by this project's standards and comfortably outside the ± 0.02 noise band on a *directional* basis, since every step goes the same way. Both generated sets go the other way, and the hard set falls monotonically across the whole range.

That pattern is the diagnosis. Public sessions draw their hidden targets with a popularity-weighted sampler, so a stronger popularity prior is partly *predicting the sampler* rather than predicting the customer. dev and holdout are two slices of the same 200 sessions and share that bias; the generated and hard sets are built differently and do not. When a change gains only on the splits that share a known bias, the gain is the bias.

The hidden 800 sessions decide the real score, so 0.4 stays. Rows pop070, pop100, pop140, pop180 and pop250 are kept in tools/sweep.py so the trade-off can be re-run rather than re-argued — the same treatment weights_argmax already gets.

A rival submission scoring 0.9748 on this evaluator does run a strong popularity prior (weight 1.0 against a coverage term of 1.5) and reaches MTTC 2.19 largely because of it. This sweep is the reason we do not simply copy it: the part of that design that transfers is the sizing, and the part that does not is the prior.

The reranker does not run on turn 1

Found while tracing the dense term, and larger than that result. `rerank()` returns its input untouched when `state.query_spans()` is empty (`src/rerank.py`), and `query_spans()` deliberately skips turn 1 — the opening line is the simulator's own framing, not quoted product copy (`src/state.py`). So on turn 1 **every S6 signal is inert**: span coverage, facet agreement, category overlap, the category tail, popularity and the dense cosine alike. Turn-1 ordering is whatever S5 retrieval produced, full stop.

That is measurable: with the dense term on, the target's rank changes on turn 1 in **zero** of 200 sessions, while it moves on 16.7% of later turns.

Two things follow. It explains why §S5's category pool moved turn 1 so far ($0.9235 \rightarrow 0.9346$ on its own) while rerank-side weight tuning never could — retrieval is the only stage that runs there. And it means the agent's single most valuable turn, the one §S7's sniper sizing stakes a whole slate on, is ranked by the least evidence-aware stage in the pipeline. Making the reranker run on turn 1 — with the opening's category words treated as the weak evidence they are, rather than skipped — is the most promising untried lever left.

The two optional layers, re-measured on the merged agent

Since removed. The dense half of this section is a record, not a description of the code: `src/embed.py`, `tools/build_embeddings.py`, the `dense_*` sweep rows and the `RerankConfig / RetrievalConfig` dense knobs were deleted in change 20 on the strength of exactly these numbers. The LLM layer is unchanged and still ships off by default. Everything below is left as measured.

Both optional signals — the dense sentence-embedding term (§S5/§S6) and the opt-in LLM listwise reranker (§S6) — were measured before changes 18 and 19. Both were re-run afterwards, with the encoder artifact rebuilt and a live API key, and both were verified actually firing first (`EmbeddingIndex.available` and `LLMReranker.available` both `True`) — a missing artifact or key makes either layer a silent no-op that scores *identically to the baseline*, which is indistinguishable from "measured and did not help" unless you check.

Public set, 200 sessions, one process:

configuration	Hit@10	MRR	MTTC	Score	vs baseline
neither (ships)	1.000	0.961	2.67	0.9550	—
dense embedding term	1.000	0.964	2.73	0.9546	-0.0004
LLM rerank, gated	1.000	0.967	2.71	0.9558	+0.0008
both together	1.000	0.962	2.73	0.9539	-0.0011

Every row is within 0.002 of the baseline, and both layers raise MRR while losing more to MTTC than they gain — they reorder the head slightly better and take an extra fraction of a turn to do it.

The prediction going in was that both would be worth **less** than before, and they are. Both fire on *ambiguous* pools — the LLM gate is literally `state.leader_margin < 0.05` — and change 19 removed most of that ambiguity by taking turn-1 recall from 80.5% to 100% and public MRR from 0.881 to 0.961. The LLM layer was worth $+0.003525$ on the older agent; it is worth $+0.0008$ on this one. The cheap deterministic fix took the ground the expensive layer was standing on.

Two results are firmer than the public column:

- **The LLM row is not reproducible to the precision of its own effect.** The same configuration scored 0.9567 and 0.9558 on two runs. The API is not deterministic at `temperature=0.0`, so its run-to-run spread (0.0009) is the size of the effect being claimed. On holdout it is -0.0005 . There is no measurement here that survives its own noise.

Traced, the layer is not broken — it is starved. Instrumenting the gate and the calls over 40 sessions: the gate is **evaluated** only 59 times across ~110 turns (the missing ~40 are turn 1, where the reranker does not run at all — see above), **opens** 27 of those 59, and every one of the 27 calls succeeds: valid JSON, usable ids, 22 of them reordering something. But only **4 of 27 change the top-1** — and §S7's sniper sizing shows a *single* candidate on turns 1-4, so reordering positions 2-8 of an 8-item window is invisible to the score by construction. Three filters multiply: inert on turn 1, gate shut on half the rest, and only position 1 ever shown.

Across the full 200 sessions the target's rank moves on 18 of 531 turns (3.4%) — **9 up and 9 down**, mean +0.01; session outcomes 2 better, 6 worse. A coin flip, for the same reason the dense term loses: the customer quotes verbatim spec strings from the target's own metadata, so deciding between two products that both contain "Imported; Zipper closure" is not a language problem. The model reads the same product text the IDF span matcher reads and has no extra information to be smarter with.

Prompt truncation was checked as a possible cause and mostly ruled out: 86% of hard constraints fall inside the 220-character candidate window (median match position 80, median product blob 678 chars); only 6.5% are cut off. Worth widening, but not the reason.

The useful conclusion is about *placement*, not about the model. We pointed a language model at the one job on this task where exact string matching is already optimal. Its comparative advantage is understanding a **reworded** customer — parsing free text into constraints, or choosing the next question — which is where tools/stress_harness.py shows the agent actually losing, and which is where the strongest published submission on this evaluator points its own model.

- **The dense term is negative exactly where it was supposed to help.** It exists for paraphrase recall — the one signal that scores meaning rather than exact tokens. On the hard set it is -0.0046 and under the paraphrase:heavy+browse-gated stress customer -0.0048. Traced against the same run with the term off, over 527 comparable turns: the target's rank is **unchanged on 83.3%**, pushed **down on 10.6%** and up on only **6.1%** (mean +0.39 ranks, downward); session outcomes 10 better, 16 worse.

The reason is not the query text, which was the first guess: encoding the compact active slots instead of the full conversation (dense_rr_slots, boilerplate stripped) scores 0.9538, worse than the 0.9546 with it, and lowering the weight to 0.2 returns 0.9556 — the optimum weight is zero.

The reason is what the customer says. The evaluator builds constraints by regex over the target's *own* metadata and quotes them verbatim, so they are strings like "Imported; Rubber sole." and "78% Cotton, 20% Polyester, 2% Elastane; Imported." — **exact identifiers, not descriptions**. A sentence embedding is built to make near-synonyms collide, which is precisely the wrong operation on a unique literal string. In public_0031 the customer quotes that fabric composition, which is nearly a fingerprint for one product; the dense term blurs it into "skinny jeans", promotes Levi's Women's Slimming Skinny Jeans to rank 1 and demotes the actual target to 2. In public_0016, "Imported; Rubber sole" encodes as *rugged boot* and the most prototypical combat boots displace the target. The term substitutes **topical** similarity for **literal** discrimination — and topical similarity is what §S5's category pool already supplies, more precisely and for free. It is competing with change 19 and losing.

This is the second independent semantic reranker to fail this way: §6's cross-encoder (change 11) concluded "the optimum semantic weight is zero" for the same reason, on a different model family. If the hidden set paraphrases heavily the trade could invert — that is what the term was insurance for — but nothing we can measure shows it paying, and an insurance policy that is measurably negative on both paraphrase sets is not insurance. Change 20 stopped paying the premium: the term is deleted rather than carried at weight zero, on the same rule that deleted span rarity (§S6) and the cross-encoder (change 11) — a code path no measurement justifies does not earn its place. Reviving it means restoring it from history, not flipping a flag.

The LLM layer stays off by default, which also keeps the submission's offline, zero-token guarantee intact — now structurally, since with the encoder gone the scoring path has no third-party dependency left to fail on.

A bug found while measuring, worth fixing before anyone enables the layer: src/llm.py never reads the usage object the API returns, and starter/agent.py reports {"prompt_tokens": 0, "completion_tokens": 0}

unconditionally. With the LLM layer on, the agent spends tokens and reports zero — and `reported_token_usage` is one of the metrics the competition collects (`docs/evaluation_config.json`). It is harmless today because the layer ships off and the offline path genuinely uses no tokens, but it would be a misreport the moment anyone turns it on for a scored run.

7. Results

How the score was built up

Stage of work	Hit@10	MRR	MTTC	Score
Supplied baseline	0.125	0.068	9.81	0.1067
+ dialog state and asking questions (§S3, §S4)	0.870	0.694	4.17	0.7799
+ verbatim span reranking (§S6)	0.940	0.786	3.58	0.8543
+ confidence gating and policy fixes (§S7)	0.940	0.791	3.41	0.8592

(An early prototype of the state-and-questions step measured 0.7811 before the code was reorganised into modules; 0.7799 is the same policy re-measured in the final structure. The small difference comes from boilerplate word handling added in §S1.)

*(This table is the historical build-up through §S7's confidence gating and is not re-measured on every change. The current figure is **0.940083** on the public set — Hit@10 1.000, MRR 0.9339, MTTC 3.005 — recorded in `results.json` and reproduced by `python3 -m evaluator.local_evaluator`. The per-change ledger from here to that number is `docs/team/agent_changes.md`.)*

Dev versus holdout

Split	Sessions	Hit@10	MRR	MTTC	Efficiency	Score
Public (all)	200	0.940	0.791	3.41	0.760	0.8592
Dev (tuned on)	120	0.950	0.814	3.27	0.772	0.8738
Holdout (untouched)	80	0.925	0.756	3.60	0.740	0.8374

Dev is higher than holdout, which is expected and healthy — dev is where the tuning happened. The gap of 0.036 is within what 80 sessions can produce by chance. The holdout number is the honest one to quote, and the one the regression test guards.

By scenario

Scenario	Sessions	Hit@10	MRR	MTTC
boundary	10	1.000	0.846	4.00
browsing	80	0.963	0.811	3.08
buying	80	0.938	0.794	3.21
intent_override	30	0.867	0.713	4.60

Intent override is the weakest scenario and the clearest target for further work. Part of the gap is structural — those sessions cannot convert before the override arrives on turn 3 or 4 (consequence #5 in §3), which puts a floor under MTTC. The rest is genuine, and §S3's ideas are aimed at it.

Operational

Model / API	None. No LLM, no network, no credentials.
Dependencies	Python standard library only
Cost	\$0.00
Tokens	0 prompt, 0 completion
Index build	4.13s, once at startup
Per-turn latency	16.6ms mean, 19.8ms p95, 21.0ms max
Peak memory	~282 MB
Full 200-session evaluation	~28s

The agent behaves identically with the network disabled, because there is no online path to fall back from. This was deliberate: the submission rules reserve the right to score under network restrictions, and an agent scoring zero in that environment is worth less than one scoring 0.8592 everywhere.

8. Why upstream Amazon data was ruled out

The catalog derives from the public Amazon Reviews 2023 dataset, which contains far more than the 50,000 frozen products. Using it was considered and **deliberately rejected**. The reasoning, in order of strength:

1. It cannot add information to the channel that matters. This is decisive. Every constraint the simulated customer can ever utter is built by `intent_card()` (`evaluator/local_evaluator.py:52`) from `features`, `details`, `price`, and `searchable_text()` over `title / features / details / description / categories / store` (`evaluator/local_evaluator.py:22`). **All of those fields are already complete in** `data/catalog.jsonl`. The process generating the dialogue is closed over the frozen catalog, so upstream data cannot tell us more about *what the customer will say* — and interpreting what they say is the entire task.

2. It does not address the measured bottleneck. Retrieval already recalls the target 80/80 at median rank 1 (§5). The losses are in ranking near-identical candidates. More product text does not break those ties.

3. It could actively hurt. The traced belt failure was *too many candidates matching generic phrases*. Richer descriptions give non-target products more surface on which to match. And we have direct evidence of this class of signal backfiring: the popularity prior had to be held to weight 0.02 (§6) because the target is one specific purchase, not a bestseller.

4. Rule risk with a bad asymmetry. `docs/competition_specification.md:13` places "private-label reconstruction" and "identifiers outside the frozen catalog" out of scope, and the organizer deliberately stripped review text, user IDs, timestamps and purchase history for anonymisation. Re-attaching that data by `parent_asin` works against stated intent even where it is not literally banned — risking a 0.86 result for a marginal gain.

5. Disproportionate cost. Tens of gigabytes of download plus a distillation script, producing an asset that must then be committed to preserve the no-network guarantee. It breaks the one-command reproduction story for very little.

The honest hole in this argument: `materialize_hidden_fields()` (`evaluator/local_evaluator.py:204`) short-circuits when a session already carries an `intent_card`, so the private sessions *could* in principle ship cards built from richer data than the frozen catalog. That seems unlikely given the organizer froze the catalog as the shared artifact, but it cannot be verified from here.

The legitimate half, kept. The one genuinely useful upstream idea — mining real attribute vocabularies instead of hand-writing keyword lists — can be done against the frozen 50,000-product catalog itself. Same benefit, no download, no rule risk. It appears in §10.

9. Files at a glance

File	Stage	What to look at
starter/agent.py	adapter, S7, S9	respond() for the whole turn flow; _confident() for timing
src/index.py	S1	CatalogIndex._build(); DEFAULT_WEIGHTS at line 25
src/text.py	S1	constraint_spans() — how customer phrases are normalised
src/router.py	S2	classify(); the docstring records the scope-reduction measurement
src/state.py	S3	DialogState.observe(); apply_override() for the deviation
src/policy.py	S4	FixedPolicy (line 40, ships); InfoGainPolicy (line 72)
src/retrieval.py	S5	retrieve() and _rrf()
src/rerank.py	S6	rerank() — the span-coverage loop
src/facets.py	shared	extract(); currently unused by ranking (see §S6 ideas)
tools/sweep.py	S0	build_configs() — add a row to test a change
tests/	S9	test_regression.py guards the 0.78 floor
ARCHITECTURE.md	—	Data-flow diagram and stage boundaries
README.md	—	Results and decision rationale

Never modify evaluator/, data/, and the five frozen files at the root of docs/ (competition_specification.md, agent_api_contract.json, evaluation_config.json, baseline_results.json, submission_rules.md) — they are organizer-owned and scoring is invalid if they change.

10. Cross-cutting and orchestration ideas

These are not tied to one stage — they concern how the stages work *together*, or they touch several at once.

Orchestration

Close the loop between asking and ranking. Right now the pipeline is a one-way pass: state -> retrieve -> rerank -> ask. The clarification policy reads the candidate pool, but nothing feeds *ranking confidence* back into *what to ask*. A genuinely adaptive agent would notice "my top two candidates differ only in colour" and ask about colour specifically — a question chosen to break the current tie rather than to split the pool in general. This is the most interesting unbuilt idea in the project, it needs no new data, and it connects S4, S6 and S7 into an actual feedback loop rather than a pipeline.

Adaptive orchestration by session state — scaffolded, not wired in (PR #7). Every session still runs the identical sequence of stages. src/context_programming.py adds an AdaptiveOrchestrator.align_strategy() that reads exactly the signals this idea named (productive_turns, dead_attributes, plus candidate-pool entropy) and classifies the turn into a DialogPhase — exploring, converging, override_reversal, stagnating. respond() calls it every turn and keeps the result in a local named plan, but nothing downstream reads plan: _shortlist() and _confident() still only consume is_buying and the two confidence-margin config fields directly (see §S7, "Track-aware margin"). Confirmed by ablation in docs/team/agent_changes.md change 14 — collapsing the buying-track margin back to the default reproduces the pre-PR score bit-for-bit even though align_strategy() still runs every turn. So the diagnosis this idea called for now exists in code; the part still open is acting on it — routing stagnating sessions to a different strategy, or gating on recommendation_cutoff instead of the flat turn number,

would be new score-moving territory this module already computes the inputs for.

A per-turn budget. All stages run every turn regardless of whether they can contribute. The focused retrieval route only matters after an override; reranking only matters once spans exist. Skipping stages that cannot help would cut latency, and more importantly would make the agent's reasoning legible: "I skipped reranking because nothing has been disclosed yet."

Explain the recommendation. The reranker knows exactly why each candidate scored well — which phrases matched. Surfacing that in the customer-facing message ("these all have a stainless steel band and a date window") costs nothing, is listed under Innovation Directions in docs/competition_specification.md:87, and makes the demo far more compelling than a bare list.

Cross-cutting

Learn the weights instead of setting them. At least eight constants across the pipeline were set by hand or inherited: `DEFAULT_WEIGHTS` (S1), `weight_focused` (S5), `PRE_OVERRIDE_WEIGHT` (S3), `span_weight / retrieval_weight / popularity_weight / length_bonus` (S6), `confidence_margin` (S7). There are 176 known-correct public sessions to learn from. A single offline fitting pass — plain logistic regression, entirely within the standard-library and no-network constraints — would likely beat hand-tuning across the board and would be more defensible than a series of individual sweeps. **This is the highest-expected-value cross-cutting change.**

Mine facet vocabularies from the frozen catalog. `src/facets.py` uses hand-written keyword lists for material, colour, style, size and use case. Harvesting these from the 50,000 products themselves would be broader, less arbitrary, and would improve both the clarification policy (S4) and the proposed facet-agreement ranking signal (S6). This is the legitimate half of the rejected upstream idea, with none of its downsides.

An error-analysis tool. The remaining 6% of sessions that miss have never been examined as a group. A script that dumps every missed session with its disclosed constraints, the target's actual rank, and what outranked it would likely reveal one or two systematic causes worth more than any parameter sweep. The belt case in §S6 was found this way, by hand, and immediately suggested category anchoring.

Make robustness measurable, not assumed. The no-network guarantee, the never-raises guarantee and the latency budget are all currently claims backed by design rather than by continuous verification. Tests that actively enforce them (§S9 ideas) would turn three assertions in this document into three facts.

Glossary

Term	One-line meaning
BM25	Text-search scoring formula. Rare words count more; matches in short important fields count more.
Constraint span	A short phrase the customer said, normalised so it can be matched against product text.
Dev / holdout	120 sessions used for tuning / 80 kept untouched to check the tuning generalises.
Entropy	A measure of how evenly a set is split. High entropy = very uncertain = a useful question to ask.
Facet	A structured product attribute with limited values — material, colour, price band.
FTS5	SQLite's built-in full-text search engine. Requires no installation.
Gain ratio	Entropy divided by its maximum, to stop attributes with many values from always winning.
Hit Rate@10	Fraction of sessions where the target appeared in a shown top-10 list.

IDF	Inverse document frequency — the "rare words matter more" weighting inside BM25.
MRR	Mean Reciprocal Rank — average of $1/\text{position}$. Rewards ranking first, not merely appearing.
MTTC	Mean Turns To Conversion — average turn on which the hit happened; a miss counts as 11.
Overfitting	Tuning until you fit the noise in your test data, so results don't transfer to new data.
parent_asin	The product ID. The only thing scored, and only on exact match.
Provenance	Metadata about <i>when and under what circumstances</i> a piece of information arrived.
Reciprocal rank	$1 / \text{position}$. Rank 1 -> 1.0, rank 10 -> 0.1, miss -> 0.
Reranking	Reordering an existing shortlist to put the best candidate first.
Retrieval	Finding a shortlist of candidates from the full catalog.
RRF	Reciprocal Rank Fusion — combining ranked lists using positions rather than scores.
Stopword	A common word removed before searching because it carries no signal.
Target	The hidden product the agent must find.
Top-k	The first k items of a ranked list; here $k = 10$.